



INSTITUTE OF INFORMATION TECHNOLOGY
UNIVERSITY OF DHAKA



TERRA-AUDIT

*– AI-Assisted Digital MRV Platform for AWD Rice Irrigation and Agricultural Land
Management Carbon Credits*

Submitted As

Technical Report

SE801 Project Final Technical Report

Submitted By

Kazi Nahid

BSSE Roll: 1437

Supervised By

Dr. Emon Kumar Dey

Professor

Institute of Information Technology

University of Dhaka

Submission Date

Oct 10, 2026

Letter of Transmittal

October 10, 2026

BSSE 4th Year Exam Committee

Institute of Information Technology

University of Dhaka

Subject: Final Report Submission — Terra-Audit: AI-Assisted Digital MRV Platform for AWD Rice Irrigation and Agricultural Land Management Carbon Credits

Dear Sir,

I am submitting the technical report for my Software Project Lab 3 project, Terra-Audit: AI-Assisted Digital MRV Platform for AWD Rice Irrigation and Agricultural Land Management Carbon Credits. This report covers the project's requirements, software design, implementation, interface design, testing, user guidance, and the limitations of the current system. I welcome your feedback and suggestions for further improvement.

Thank you for your consideration.

Sincerely,

Kazi Nahid

BSSE-1437

Institute of Information Technology

University of Dhaka

October 10, 2026

Supervisor's Signature

Acknowledgement

I am grateful to the Institute of Information Technology, University of Dhaka, for giving me the opportunity to work on the Software Project Lab 3 project “Terra-Audit: AI-Assisted Digital MRV Platform for AWD Rice Irrigation and Agricultural Land Management Carbon Credits.”

I would like to thank my supervisor, Dr. Emon Kumar Dey, Professor, Institute of Information Technology, University of Dhaka, for the guidance and feedback given at every stage of the project, from requirements analysis through design and implementation to evaluation. That feedback shaped how the project developed.

Lastly, I would like to thank my classmates for their helpful suggestions and feedback during the project reviews, and my family for their constant encouragement.

Abstract

Alternate Wetting and Drying (AWD) irrigation in rice and improved agricultural land management (ALM) on cropland both reduce greenhouse-gas emissions. Proving the reduction is the hard part: the monitoring, reporting and verification (MRV) evidence behind a carbon claim is usually spread across field logs, spreadsheets and one-off calculations that are difficult to trace. Terra-Audit is a web platform that keeps this evidence in one place so that a project can be prepared for audit. It supports two accounting pathways: Verra VM0051 v1.1 for AWD rice, and VM0042 v2.2 for cropland ALM with a restricted VMD0054 v1.1 leakage adapter. Each pathway implements a defined subset of its methodology, not all of it. The system is built with Next.js, FastAPI and a database-backed background worker. Users register fields, record evidence, run Sentinel-1 analysis through Google Earth Engine and save each calculation as a fixed version that can then go through internal review. Machine-learning detectors, crop models and an LLM explanation feature are included only as aids; carbon values and readiness states are always computed by server code.

In the final test run on 7 October 2026, all 177 backend tests passed, as did the frontend tests, the production build, the TypeScript check and lint (0 errors). Field validation of the AWD detector, full methodology coverage and verification by an external VVB or Verra were outside this project; Chapter 14 discusses what that leaves open.

Keywords: digital MRV, AWD rice, VM0051, VM0042, carbon accounting, evidence provenance, satellite monitoring, readiness, internal review.

Table of Contents

1. Introduction.....	1
1.1 Project Title.....	1
1.2 Problem Statement and Motivation.....	1
1.3 Objectives.....	1
1.4 Scope.....	2
1.5 Deliverables.....	3
1.6 Project Timeline.....	3
1.7 Organization of the Report.....	4
2. Background and Related Work.....	4
2.1 AWD Irrigation and Rice Methane.....	4
2.2 Satellite Monitoring of Rice Fields.....	4
2.3 Carbon Methodologies and Digital MRV.....	4
2.4 Gap Addressed by Terra-Audit.....	4
3. Requirements Analysis.....	5
3.1 Requirements Inception.....	5
3.2 Functional Requirements.....	5
3.3 Non-functional Requirements.....	6
3.4 Stakeholders and Viewpoints.....	7
3.5 Quality Function Deployment.....	8
3.6 Usage Scenarios and User Stories.....	8
4. Scenario-Based Modeling.....	9
4.1 Use Case Diagrams.....	9
4.1.1 Use Case Level 0: System Context.....	9
4.1.2 Use Case Level 1: Modules of Terra-Audit.....	10
4.1.3 Use Case Level 2: Carbon Calculation.....	11
4.1.4 Use Case Descriptions.....	11
4.2 Activity Diagrams.....	13
4.2.1 Cropland ALM Detailed Activity Flow.....	15
5. Data-Based Modeling.....	16
5.1 Entity-Relationship Diagram.....	16
5.1.1 Relationship Summary.....	17
5.2 Database Schema.....	17
5.3 Dataset Description (AI-Assisted Detection).....	18
6. Class-Based Modeling.....	19
6.1 Analysis Classes.....	19
6.2 Class Cards (CRC Model).....	20
6.3 Design Class Diagram.....	20
6.3.1 Design Class to Implementation Mapping.....	21
7. Architectural Design.....	22
7.1 Architectural Context Diagram.....	22
7.2 Defining Archetypes.....	22
7.3 Top-Level Components.....	23

7.4 Mapping Requirements to Software Architecture.....	25
7.5 Technology Stack.....	25
7.6 Design Evolution and Alternatives.....	26
8. Component-Level Design.....	26
8.1 Persistent Data Sources and Interfaces.....	26
8.1.1 Context-Level Data Flow (DFD Level 0).....	26
8.1.2 Level-1 Data Flow (DFD Level 1).....	27
8.2 API Design.....	28
8.3 Behavioral Design.....	29
8.3.1 Behavioral Event Table.....	29
8.3.2 State Diagrams.....	29
8.3.3 Sequence Diagrams.....	31
8.4 Deployment Design.....	33
9. AI Engineering Design.....	34
9.1 Classical AWD ML Pipeline.....	34
9.2 Model Selection and Evaluation.....	34
9.3 Managed Crop Models.....	35
9.4 Generative AI Explanation Architecture.....	35
9.5 Prompt and Context Design.....	35
9.6 AI Authority Boundary and Evaluation Limitations.....	36
10. Interface Design.....	36
10.1 Interface Objects and Actions.....	36
10.2 Primary User Workflows.....	37
10.3 UI Events and State Transitions.....	37
10.4 Responsive, Validation and Accessibility Design.....	38
10.5 Interface States as Seen by the User.....	38
11. Implementation.....	41
11.1 Final Feature Status.....	42
11.2 Worked Calculation Checks.....	42
12. Testing.....	42
12.1 Testing Objectives.....	42
12.2 Testing Approach, Strategy and Pass/Fail Criteria.....	43
12.2.1 API Endpoint Verification.....	43
12.3 Detailed Test Cases.....	44
12.4 Verification Results.....	45
12.5 Risks and Contingencies.....	46
12.6 Requirements Traceability Matrix.....	46
13. User Manual, Deployment and Repository.....	48
13.1 End-User Workflows.....	48
13.2 Export and Troubleshooting.....	48
13.3 Local Installation and Development Setup.....	49
13.4 Deployment and Public Repository.....	49
14. Evaluation, Limitations and Conclusion.....	50
14.1 Final Evaluation.....	50

14.2 Methodology and Accounting Limitations.....	50
14.3 Scientific Validation Limitations.....	51
14.4 Provenance and Reproducibility Limitations.....	51
14.5 Security and Operational Limitations.....	51
14.6 Generative-AI Limitations.....	51
14.7 Prioritized Future Work.....	51
14.8 Conclusion.....	51
References.....	52

List of Figures and Tables

Figures

- Fig 1 Project timeline (Gantt chart) — p. 3
- Fig 2 Use Case Level 0 — Terra-Audit as a single system with its actors and external systems — p. 10
- Fig 3 Use Case Level 1 — the ten user-goal modules and their actors — p. 10
- Fig 4 Use Case Level 2 — Carbon Calculation (1.4) with «include» and «extend» relationships — p. 11
- Fig 5 Overall activity diagram with swimlanes for the project user, the Terra-Audit server and the durable worker — p. 14
- Fig 6 Activity diagram — methodology readiness and internal review with swimlanes — p. 15
- Fig 7 Entity-relationship diagram (crow's-foot notation) of the core persistent entities — p. 16
- Fig 8 Entity-relationship diagram of the calculation, readiness and internal-review tables — p. 17
- Fig 9 CRC cards for the nine core design classes — p. 20
- Fig 10 Design class diagram: principal classes and modules, key operations and dependencies — p. 21
- Fig 11 Architectural context diagram (ACD) of Terra-Audit — p. 22
- Fig 12 Layered architecture archetype instantiated for Terra-Audit — p. 23
- Fig 13 Pipe-and-filter archetype: the Sentinel-1 signal-processing pipeline of the rice AWD pathway — p. 23
- Fig 14 Top-level component diagram of Terra-Audit — p. 24
- Fig 15 Level-0 data-flow diagram — p. 27
- Fig 16 Level-1 data-flow diagram: processes, data stores and external flows — p. 28
- Fig 17 State diagram — durable background job lifecycle (src/jobs/queue.py) — p. 30
- Fig 18 State diagrams — (a) calculation version lifecycle and (b) data-quality issue lifecycle — p. 30
- Fig 19 State diagram — internal review submission lifecycle (src/projects/reviews.py) — p. 31
- Fig 20 Sequence diagram — calculation commit from authenticated request to immutable version — p. 31
- Fig 21 Sequence diagram — Sentinel-1 signal run through the durable job queue — p. 32
- Fig 22 Sequence diagram — bounded AI explanation with server-side validation — p. 33
- Fig 23 UML deployment diagram of Terra-Audit — p. 34
- Fig 24 AI authority and data flow — p. 36
- Fig 25 Application shell and project navigation — p. 38
- Fig 26 Monitoring batch state: one running batch with two pending child jobs; no data-quality issues — p. 39
- Fig 27 Rice calculation readiness and inputs — p. 40
- Fig 28 Review-related states — p. 41
- Fig 29 AI workspace — crop-model controls — p. 41

Tables

- Table 1 Functional requirements and priorities — p. 5
- Table 2 Non-functional requirements and verification targets — p. 6
- Table 3 Stakeholders and viewpoints — p. 7
- Table 4 Use-case inventory and actor mapping — p. 11
- Table 5 Detailed use-case specifications — p. 11
- Table 6 Persistent database schema by domain — p. 18
- Table 7 Analysis-class identification from scenario nouns — p. 19
- Table 8 Design classes and their implementing modules — p. 21
- Table 9 Component responsibilities — p. 24
- Table 10 Mapping functional requirements to architectural elements — p. 25
- Table 11 Technology stack — p. 25
- Table 12 REST API summary by router group — p. 28
- Table 13 Behavioral event table — p. 29
- Table 14 User-interface objects and actions — p. 37
- Table 15 UI events and resulting state changes — p. 37
- Table 16 Direct API endpoint verification — p. 43
- Table 17 Detailed test cases with outcomes — p. 44
- Table 18 Verification results (7 October 2026, commit bc402f2) — p. 45
- Table 19 Functional requirements traceability matrix — p. 46
- Table 20 End-user workflows — p. 48
- Table 21 Export options and troubleshooting — p. 49

1. Introduction

1.1 Project Title

Terra-Audit — AI-Assisted Digital MRV Platform for AWD Rice Irrigation and Agricultural Land Management Carbon Credits.

1.2 Problem Statement and Motivation

Flooded rice paddies are a major source of agricultural methane. AWD irrigation is a known way to reduce it, but showing that a farm actually practised AWD, and turning that into a carbon number someone can audit, is hard to do at scale. Farmer irrigation logs are often incomplete. Installing sensors in thousands of smallholder plots is expensive. And when the accounting is done in a spreadsheet, the final figure tends to lose its link to the evidence, the assumptions and the methodology version behind it. Cropland ALM projects have a similar problem: practice changes, production data, soil organic carbon (SOC) samples, leakage and uncertainty all come from different sources and have to stay consistent from one verification period to the next.

Terra-Audit was built to keep that evidence in one place. For rice fields it uses Sentinel-1 SAR data from Google Earth Engine to look for flooding and drydown. For cropland fields it stores practice, soil, laboratory, production, livestock and leakage records. On top of these records the platform runs the carbon calculations, checks the methodology requirements, saves each result as a fixed version, supports an internal review step and produces exports. Satellite and AI results count as supporting evidence only; they do not replace field validation or verification by a VVB.

1.3 Objectives

- Use free Sentinel-1 SAR data to detect flooding and drydown in rice fields at scale, while making it clear that these are satellite estimates and not validated AWD ground truth.
- Support cropland ALM projects under a defined subset of Verra VM0042 v2.2, using practice, production, livestock, soil-sampling, laboratory, custody and reviewed SOC records instead of satellite signals.
- Implement carbon calculations for the supported parts of VM0051 v1.1 (rice AWD, QA3/default-factor pathway) and VM0042 v2.2 (cropland ALM), and show the intermediate values and assumptions so that a reviewer can follow them.
- Model the evidence lifecycle directly: crop seasons and their corrections, observations, practice events, attachments, production records, soil sampling, laboratory results, chain-of-custody records, reviewed SOC inputs and integrated crop-livestock data.
- Build methodology bundles, corrections, applicability and readiness checks, plus a restricted VMD0054 v1.1 leakage module, so that a missing or unsupported requirement is reported to the user instead of being quietly counted as met or as zero.
- Store every calculation as a numbered version with frozen evidence, inputs and results, and let an eligible version go through internal review.
- Include Random Forest/XGBoost detectors, crop models and LLM draft explanations as experimental aids: a person reviews their output, and none of them can change a calculated value.
- Deliver a multi-user web application with organization and project access control, background jobs, calculation history, review and export, and be open about where security, provenance and methodology coverage are still incomplete.

1.4 Scope

In scope:

- Accounts, organizations, projects, farms and fields behind authentication. Fields are registered from the supported geometry inputs, and the server decides which organization a request belongs to.
- Two accounting pathways, rice_awd and cropland_alm_vm0042. A field's type decides which pathway it follows and cannot be changed after registration.
- Rice signal analytics on Sentinel-1 VV/VH observations: compatible cached series are reused, other requests run as durable signal jobs, and the smoothed series goes through threshold-based AWD and phenology logic. Experimental ML detectors are optional.
- Structured cropland evidence for VM0042: crop seasons, practice schedules, production records, soil sampling plans with their strata and samples, laboratory results, custody history, reviewed SOC mapping and the supported non-SOC emission inputs.
- Baseline and project livestock schedules, within the supported integrated crop-livestock scope.
- The livestock emission terms supported by the scoped ALM engine, including selected enteric-fermentation and pasture/range/paddock manure sources.
- Leakage assessment with the restricted VMD0054 v1.1 adapter, based on structured production evidence. Each assessment is tied to a project, field, methodology bundle and reporting period.
- A versioned calculation history. Each version freezes its inputs, results, readiness state and methodology bundle, and records which earlier version it supersedes. Eligible versions can enter the internal review workflow.
- Management of methodology documents and bundles, corrections and project applicability, together with deterministic readiness evaluation, findings and the reviewer workflow.
- Deterministic accounting for the supported parts of VM0051 v1.1 (rice) and VM0042 v2.2 (ALM), including the implemented uncertainty, leakage, SOC and selected non-SOC terms.
- Classical AWD ML, crop-model experiments and LLM draft explanations. None of these feeds into the official calculation or into any verification decision.
- Exports: legacy verification PDF/JSON/CSV, versioned calculation JSON and an ALM calculation PDF built from the saved snapshot. The application also has private evidence attachments, calculation history and admin pages for jobs, methodology sources, AI providers and worker status.
- The rice pathway targets Bangladesh and South Asia. SQLite is used for local development and tests; hosted deployments use PostgreSQL with local or private S3-compatible storage.

Out of scope (this phase):

- Field-level AWD ground-truth validation and on-field sensor integration. The satellite detector is still an experimental source of supporting evidence.
- Methodologies other than the implemented parts of VM0051 and VM0042, and full coverage of every requirement, tool, safeguard and procedure in those methodologies and in the VCS program.
- Validation or verification by a VVB, Verra registry integration and VCU issuance. A quantity calculated in Terra-Audit is not an issued carbon credit.
- Production security certification, including full project-level authorization coverage, database foreign keys and row-level security, penetration testing, tested backup and restore, and sealed provenance for every legacy export path.

1.5 Deliverables

- A Next.js frontend and FastAPI backend covering projects, fields, evidence, monitoring, calculations, review, AI, exports and administration. The earlier Streamlit app has been retired.
- A persistence layer, organized by domain, that runs on SQLite or PostgreSQL. It holds structured records for accounts, projects, fields, evidence, soil, production, calculations, reviews, methodologies, jobs and AI, plus private storage for attachments and model files.
- Calculation engines for the supported parts of VM0051 v1.1 and VM0042 v2.2, with the restricted VMD0054 v1.1 leakage module, readiness gates and versioned storage of results.
- A background worker and job queue for signal and AI tasks, the Sentinel-1 threshold pipeline, experimental RF/XGBoost and crop models, and the provider layer for LLM explanations.
- Evidence management, private attachments, calculation snapshots and history, internal review with findings, and the export formats listed in Section 1.4. Known provenance gaps in the legacy exports are documented.
- Methodology source and bundle management, applicability and readiness checks, history and portfolio views, operations monitoring, and AI explanations that the server checks before display.
- This report and user manual, the public GitHub repository, and a test baseline from 7 October 2026 (commit bc402f2): 177 backend tests passed with 0 failures, the frontend tests passed (4/4), and the production build, TypeScript check and ESLint (0 errors, 5 warnings) all succeeded.

1.6 Project Timeline

Fig 1 compares the planned schedule from the SRS and midterm report with the actual windows reconstructed from the repository history.

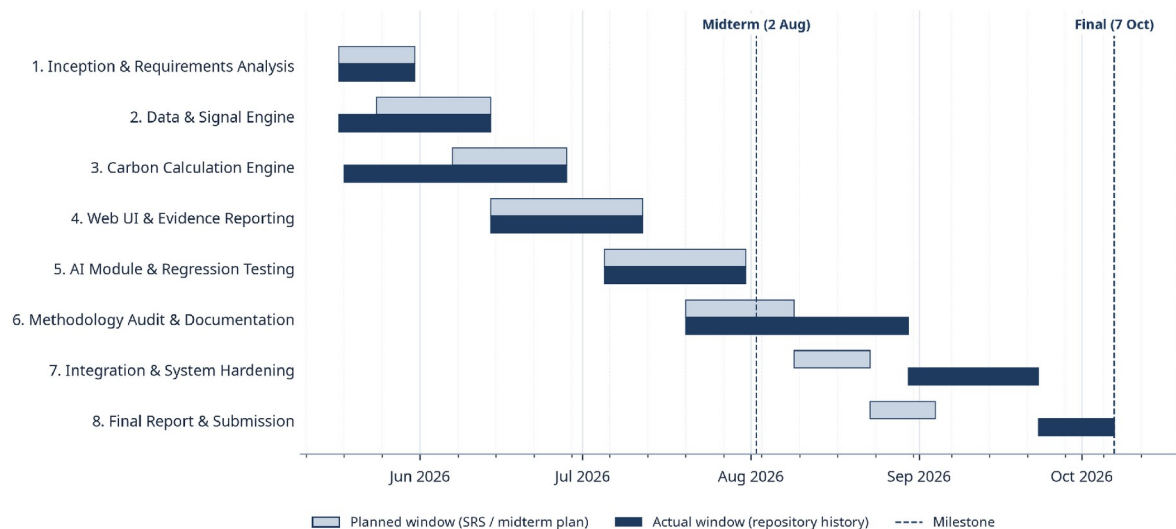


Fig 1: Project timeline (Gantt chart): planned windows versus actual windows, with the midterm (2 Aug 2026) and final verification (7 Oct 2026) milestones.

Phases 1–5 (requirements, signal engine, VM0051 engine, web UI and AI module) finished roughly on schedule and were shown at the 2 August midterm; phases 2 and 3 actually began early, during inception. After the midterm the methodology work (phase 6) grew much larger than planned, as methodology bundles, corrections, readiness checks and the whole VM0042/VMD0054 ALM pathway were added. Integration, hardening and the final report therefore slipped into September and early October. That extra time produced the multi-user system, background jobs, versioned snapshots and internal review described in this report.

1.7 Organization of the Report

Chapter 2 reviews background on AWD, SAR-based rice monitoring and the Verra methodologies. Chapters 3–5 present the requirements, scenario-based models and data-based models. Chapters 6–8 cover class-based modeling, architectural design and component-level design, including the behavioral and deployment models. Chapter 9 describes the AI engineering design, Chapter 10 the interface design, Chapter 11 the final implementation and Chapter 12 testing. Chapter 13 is the user manual with deployment and repository details, and Chapter 14 evaluates the system and concludes the report.

2. Background and Related Work

2.1 AWD Irrigation and Rice Methane

Flooded rice fields emit methane because waterlogged soil becomes anaerobic, which favours methanogenic microbes. Under Alternate Wetting and Drying (AWD) the water level is allowed to fall below the soil surface before the field is irrigated again, which interrupts those conditions; practised correctly, AWD reduces methane emissions while maintaining yield [9]. Verra VM0051 v1.1 quantifies the reduction with IPCC water-regime, pre-season and organic-amendment scaling factors [1], [4], and CH₄ and N₂O are converted to CO₂e with the AR5 100-year GWP values [5]. The QA3 default-factor approach used in Terra-Audit applies only to projects below 60,000 tCO₂e per year.

2.2 Satellite Monitoring of Rice Fields

Synthetic aperture radar (SAR) is well suited to rice monitoring: C-band backscatter responds to standing water and canopy structure, and cloud cover does not affect it. A flooded field reflects the signal away from the sensor (specular reflection), so its VV backscatter is low, and backscatter then rises as the canopy develops [13], [14]. Sentinel-1 provides free dual-polarized (VV/VH) C-band observations with a 6–12 day revisit [7], [15], and Google Earth Engine offers planetary-scale access to the Sentinel-1 GRD archive [8], [16]. Terra-Audit uses this pattern to detect flooding and drydown events, but treats the detections as supporting evidence only, because a field can be drained and re-flooded between two passes without either event being observed. Crop-season monitoring runs also store scene-level Sentinel-1 and Sentinel-2 observations (including optical indices such as NDVI) with a coverage-quality summary, but infer no crop or practice.

2.3 Carbon Methodologies and Digital MRV

VM0042 v2.2 credits improved agricultural land management through soil organic carbon (SOC) change, non-SOC emission sources and uncertainty deductions, while VMD0054 v1.1 estimates leakage from displaced agricultural production [2], [3]. MRV for such projects usually relies on spreadsheets and manually collected documents, and once a number is copied into a report it loses its link to the evidence, methodology version and calculation history behind it. Digital MRV tries to keep those links so that a result can be traced and reproduced later. Validation and verification remain the responsibility of an accredited VVB under the Verra VCS Program [6].

2.4 Gap Addressed by Terra-Audit

In practice, remote-sensing tools and carbon spreadsheets are kept apart. A reviewer may receive a credit figure in a spreadsheet, a folder of Sentinel-1 plots and a farmer's irrigation log, and then has to work out which plots and log entries the figure was based on and which methodology version was used. In Terra-Audit each saved calculation version stores its evidence snapshot and the methodology

bundle it was computed with, so the reviewer can answer those questions from the record itself instead of reconstructing them by hand.

3. Requirements Analysis

3.1 Requirements Inception

The original SRS framed Terra-Audit as a way to make agricultural MRV evidence easier to trace and less dependent on scattered manual records. At the midterm the system was still a single-user rice tool: it fetched Sentinel-1 data, detected AWD events and ran the VM0051 calculation. Adding the VM0042 pathway after the midterm changed the requirements more than any other decision. Cropland fields have no satellite signal to work from, so the system now needed practice schedules, soil sampling plans and laboratory results, production records and a leakage calculation, none of which existed in the rice design. The same change brought in methodology bundles, corrections and readiness checks, because VM0042 has many requirements that the system covers only in part, and those gaps had to be visible to the user.

Two further changes came from the move to a multi-user web application. Once several people could work on the same project, a result needed a version history and a review step, so versioned calculations and internal review were added as requirements. Earth Engine and AI calls could also take too long to run inside a web request, which added the background-job requirement. The requirements in this chapter describe the final system. Where something cannot be fully automated, such as additionality or external verification, it is marked as partial, unsupported, blocking or needing human review instead of being left out.

3.2 Functional Requirements

Table 1: Functional requirements and priorities

ID	Requirement	Priority
FR1	Provide authenticated account access and account-recovery flows; derive the active organization from the authenticated user rather than from a client-selected tenant identifier.	Must
FR2	Create and manage projects, farms, and fields; register each field with a supported geometry and an immutable accounting pathway (rice_awd or cropland_alm_vm0042), and maintain dated project/farm memberships.	Must
FR3	For rice fields, request Sentinel-1 VV/VH observations for a monitoring window, reuse compatible cached or active work where possible, and otherwise execute acquisition/processing through the durable signal-job workflow.	Must
FR4	Process rice time-series with the implemented smoothing, flood/drydown and phenology logic, and retain threshold-based detection as the deterministic reference path for experimental AWD ML models.	Must
FR5	Record crop seasons with correction/version history and associate field observations, practice events, monitoring runs, reviews, data-quality issues, and evidence attachments with the appropriate field/season context.	Must
FR6	For cropland ALM fields, record baseline/project practices, production records, livestock schedules, soil sampling plans, strata, samples, laboratory results, custody events, and reviewed SOC evidence required by the supported calculation scope.	Must
FR7	Calculate the implemented VM0051 v1.1 QA3/default-factor rice result deterministically, including supported CH ₄ scaling, uncertainty and irrigation-change N ₂ O terms, while exposing unsupported methodology elements separately from the numerical calculation.	Must
FR8	Calculate the implemented VM0042 v2.2 ALM subset from frozen evidence, including supported SOC, uncertainty, selected non-SOC emission terms, integrated livestock terms, and restricted	Must

ID	Requirement	Priority
	VMD0054 v1.1 displacement-leakage logic.	
FR9	Support experimental Random Forest/XGBoost AWD detectors and managed crop models without treating model predictions or model-versus-threshold agreement as independent scientific validation.	Should
FR10	Provide bundle-scoped generative AI draft explanations through configured providers, using deterministic server-built context packets and server-side validation; AI output must not control accounting values, readiness, or verification decisions.	Should
FR11	Export the formats implemented for each workflow: legacy verification PDF/JSON/CSV, versioned calculation JSON, and snapshot-backed ALM calculation PDF, while clearly distinguishing snapshot-backed outputs from legacy paths whose historical provenance is not fully frozen.	Must
FR12	Provide field/project history and portfolio views that let authorized users inspect registered fields, current methodology path, calculation versions, latest results, and relevant project context.	Should
FR13	Manage versioned methodology documents, bundles, corrections, project applicability, and deterministic readiness requirements for the supported accounting pathways.	Must
FR14	Persist each new calculation as a versioned record with frozen evidence/input/result/readiness context and supersession metadata so later evidence or calculation changes do not silently rewrite historical numerical payloads.	Must
FR15	Allow eligible calculation versions to enter an internal review workflow with submissions, reviewer assignments, review events, findings, comments, and notifications.	Should
FR16	Provide portfolio and operational views for project/calculation summaries, methodology/source coverage, configured AI providers, background jobs, and worker status.	Should
FR17	Enforce organization scoping and project-role permissions for supported workflows while allowing permitted updates to editable project/field metadata without changing immutable accounting-pathway identity.	Must
FR18	Execute long-running signal and AI work through the durable database-backed queue with worker heartbeat, retry, cancellation, and failure states rather than relying on in-request synchronous execution.	Must

3.3 Non-functional Requirements

Table 2: Non-functional requirements and verification targets

ID	Category	Requirement	Verification
NFR1	Security & Identity	Authentication secrets and credentials must be protected; production configuration must reject weak/default JWT secrets, and account tokens must be validated against the current active user and token-version state.	Authentication/account-access tests plus configuration and security-code review.
NFR2	Authorization & Tenant Isolation	Tenant identity must come from authenticated server state, and project/role authorization must be enforced for protected operations; any endpoint that remains organization-scoped rather than uniformly project-member-scoped must be documented as a limitation.	Tenant/auth regression tests and endpoint-level authorization review.
NFR3	Auditability & History	Saved calculations must retain frozen evidence/input/result/readiness context and a version chain sufficient to review historical results without silently recomputing them from current evidence.	Calculation/snapshot/export regression tests and historical-version comparisons.
NFR4	Reproducibility	Deterministic accounting and scoped fingerprints must be reproducible from stored values and stable canonical serialization; software/model versions and provenance limitations must be disclosed where complete executable provenance is not yet captured.	Golden-value tests, fingerprint tests, snapshot/version tests, and code review of version/provenance fields.

ID	Category	Requirement	Verification
NFR5	Methodology Integrity	Unsupported, incomplete, or reviewer-dependent methodology requirements must remain explicit and must not be silently interpreted as satisfied, zero, or externally verified.	Methodology/readiness tests and review of blocking/unsupported requirement behavior.
NFR6	External-Service Resilience	External-service failures (Earth Engine, email, storage, AI provider, worker availability) must degrade through actionable application/job states rather than exposing raw failures as successful results.	Job/API tests, fault-path review, and operational/manual service checks.
NFR7	Availability & Job Durability	Long-running analytics and AI work must be decoupled from request lifetimes through the durable queue, with persisted job status (pending, running, done, error or cancelled), retries where supported, and worker heartbeat visibility.	Durable-job tests, worker CLI checks, and job-state UI/API verification.
NFR8	Maintainability	Domain packages must remain separated by capability (accounts, projects, evidence, methodology, carbon, signals, jobs, AI, persistence, reporting) so API/frontend changes do not become calculation authority.	Architecture/code review plus full regression suite.
NFR9	Portability & Migration Transparency	The system must support local/test SQLite and hosted PostgreSQL configurations without changing domain behavior; deployment-specific limitations and unverified migration paths must be stated explicitly.	Fresh/repeated SQLite initialization tests, representative legacy-upgrade test, and deployment/migration review.
NFR10	Privacy & Storage	Private evidence/model artifacts must be accessed through authenticated application paths, use sanitized storage keys/checksums, and avoid exposing tenant files as public static assets; shared-storage/backup limitations must be disclosed.	Attachment/storage tests, authenticated-download review, and backup/restore script review.
NFR11	Usability & Accessibility	Important UI states—loading, queued, running, blocked, permission denied, validation failure, completed, failed and retry—must be visible and understandable without relying on color alone.	Frontend tests, ESLint, responsive/UI review, and final-state screenshot inspection.
NFR12	Scientific Validity	Satellite/ML outputs must be represented as experimental/supporting evidence unless independently validated; software tests or model-versus-threshold metrics must not be described as proof of real-world AWD accuracy.	Review of detector-label provenance plus explicit scientific-validation status in the report.
NFR13	Deployment & Operations	Supported deployment topology, external dependencies, storage/database configuration, and backup/recovery assumptions must be documented separately from what has actually been verified live.	Deployment checks, configuration review, worker/CLI checks, and explicit verified-vs-supported reporting.
NFR14	Testability	The final verification process must pass the current backend/core regression suite and frontend test/build/type checks, while any unexecuted scientific, browser-E2E, PostgreSQL, provider, or external-service validation remains explicitly separate.	Re-run on 7 October 2026: backend 177 passed, 0 failed, 0 skipped; frontend tests, build and TypeScript pass; ESLint 0 errors, 5 warnings.

3.4 Stakeholders and Viewpoints

Table 3: Stakeholders and viewpoints

Stakeholder	Interest
Carbon Project Developer / Project Lead	Creates and manages project context, registers fields, coordinates evidence, runs scoped calculations, reviews readiness/history, and prepares exports; project contributors/viewers receive narrower role-based access.

Stakeholder	Interest
Researcher / Data Scientist	Builds/evaluates experimental AWD and crop models, studies satellite/threshold behavior, and distinguishes model agreement from independently validated agronomic accuracy.
Internal Reviewer	Reviews eligible calculation versions inside Terra-Audit, records findings/comments/events, and checks evidence/readiness context; this role is internal and is not a substitute for a VVB.
Program Operator (farmer cooperative / NGO)	Coordinates enrollment and field/evidence collection across participating farms and uses the platform to prepare project records for downstream review.
Farmer / Landholder	Owns or manages enrolled land and provides or confirms agronomic, production, livestock, soil, irrigation, and other field evidence used by project staff.
System Operator / Administrator	Maintains application configuration, methodology sources, provider permissions, worker/queue operations, database/storage settings, and operational/backup tooling.
External Validation/Verification Body (VVB) / Verra Process	Independently evaluates methodology applicability, evidence, monitoring, calculations, safeguards and verification requirements outside the application. Terra-Audit internal review and estimated quantities do not constitute VVB verification or Verra-issued VCUs.
External Data and AI Services	Google Earth Engine/Copernicus provide satellite observations, while configured AI/email/storage services support optional platform functions; their availability and credentials constrain runtime behavior.

3.5 Quality Function Deployment

The QFD sorts the functional requirements in Table 1 by how users would see them. Normal requirements are the ones a project lead would ask for directly, because the system is no use for MRV preparation without them: access control, evidence, accounting, readiness, versioning, export and background jobs (FR1, FR2, FR5-FR8, FR11, FR13, FR14, FR17, FR18). Expected requirements are ones users take for granted without listing them, such as tracing a signal result back to its run, looking up earlier calculations, having a second person review a result and seeing whether jobs are running (FR3, FR4, FR12, FR15, FR16). The ML and generative-AI features (FR9, FR10) are exciting requirements: they add value when they work, but a project can be prepared without them, and they have no say in the calculated values.

Traceability runs from stakeholder need → QFD category → FR ID and priority → implementing component (Chapter 7) → verification evidence (Chapter 12).

3.6 Usage Scenarios and User Stories

US1 — Project lead: create or select a project, register its fields, add members, organize evidence and save versioned calculations, so that every result stays tied to a project and field the lead is authorized to work on.

US2 — Rice or ALM project user: collect the evidence the field's calculation needs, which is signal results for VM0051 rice or structured practice, SOC, production and leakage records for VM0042 cropland. The source of each item is kept, and incomplete requirements stay blocking.

US3 — Reviewer: inspect methodology readiness, frozen calculation versions, the evidence behind them, findings and history. Unresolved requirements must stay visible, and evidence added later must not silently change the numbers of a version that has already been saved.

US4 — Researcher or authorized AI user: run the threshold detector and the RF/XGBoost models on the same field and compare their outputs, or ask for a draft explanation of a calculation, knowing that neither result counts as field validation.

US5 — Administrator: monitor methodology sources, provider permissions, jobs and worker state, so that when a background job or external service fails, the failure can be diagnosed and is never shown as successful work.

4. Scenario-Based Modeling

4.1 Use Case Diagrams

The use-case models show what each actor uses Terra-Audit to achieve. The primary actors are project users (lead, contributor, viewer), researchers, internal reviewers and administrators. Google Earth Engine, the AI provider and the email service are external systems. The database and storage are part of the system itself, so they are not drawn as actors, and the VVB and Verra processes lie outside the system boundary.

4.1.1 Use Case Level 0: System Context

At Level 0 the whole platform is a single use case (Fig 2). Every human actor interacts with Terra-Audit, which in turn depends on three external systems.

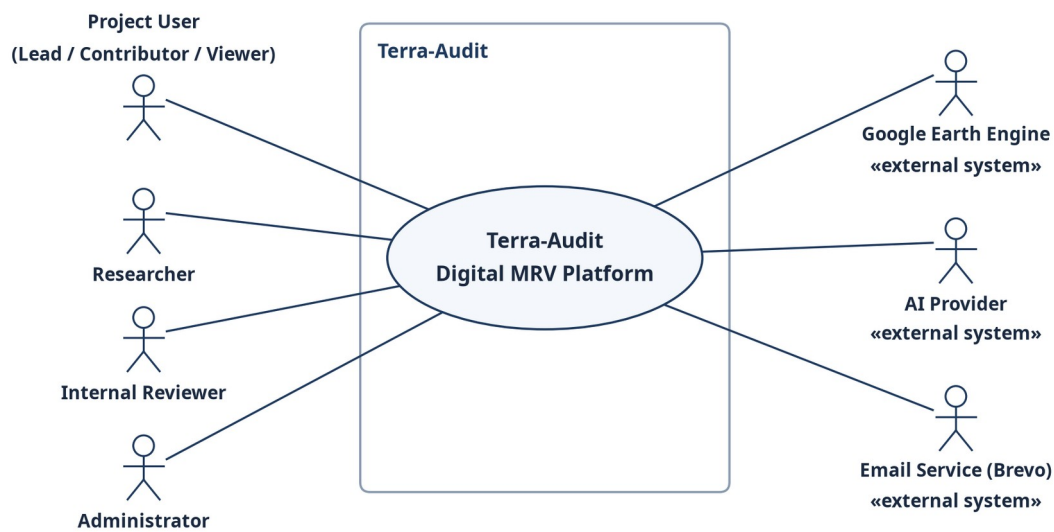


Fig 2: Use Case Level 0 — Terra-Audit as a single system with its actors and external systems.

4.1.2 Use Case Level 1: Modules of Terra-Audit

Level 1 decomposes the platform into ten user-goal modules (Fig 3); Table 4 maps each module to its actors.

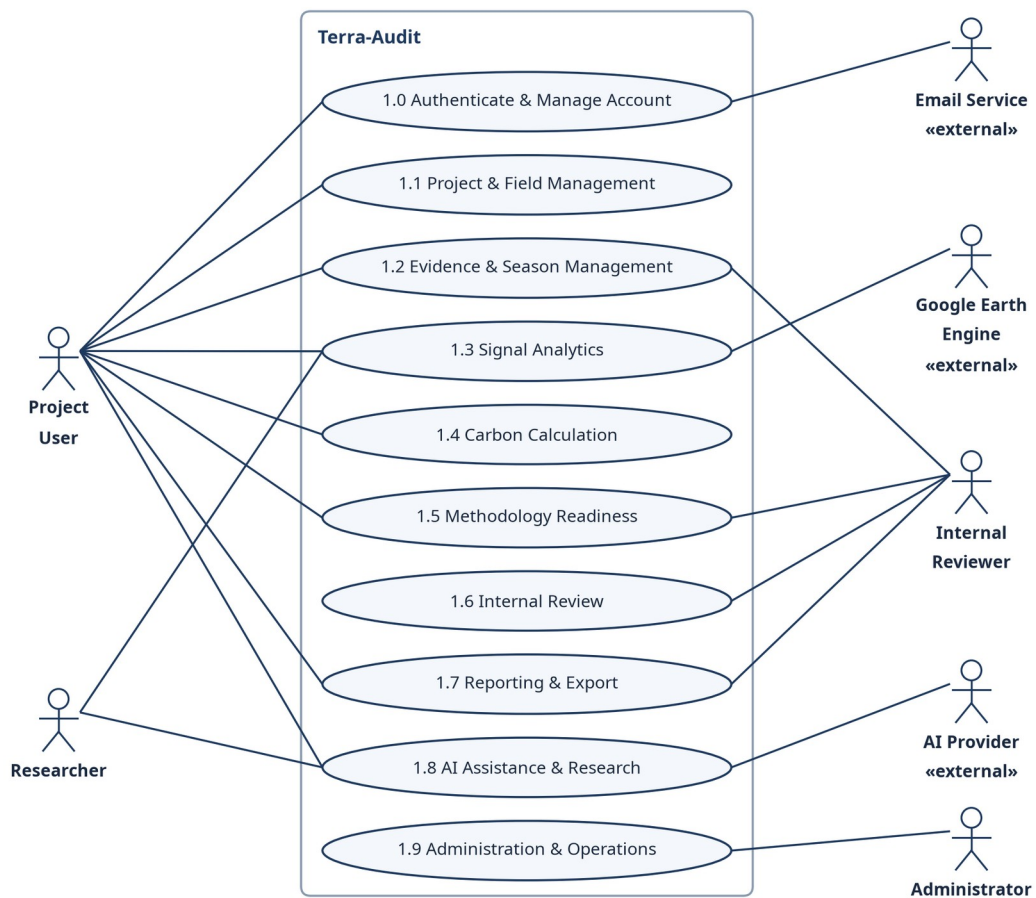


Fig 3: Use Case Level 1 — the ten user-goal modules and their actors.

Table 4: Use-case inventory and actor mapping

ID	Use Case	Primary Actor	Secondary Actor / System	Figure
L0	Terra-Audit Digital MRV Platform	All human actors	Earth Engine, AI provider, email	Fig 2
1.0	Authenticate & Manage Account	Project User, Administrator	Email service (Brevo)	Fig 3
1.1	Project & Field Management	Project Lead / Contributor	—	Fig 3
1.2	Evidence & Season Management	Project User, Internal Reviewer	—	Fig 3
1.3	Signal Analytics	Project User, Researcher	Google Earth Engine	Fig 3
1.4	Carbon Calculation	Project User	—	Fig 3, Fig 4
1.5	Methodology Readiness	Project User, Internal Reviewer	—	Fig 3
1.6	Internal Review	Internal Reviewer, Project Lead	—	Fig 3
1.7	Reporting & Export	Project User, Internal Reviewer	—	Fig 3
1.8	AI Assistance & Research	Project User, Researcher	AI provider	Fig 3
1.9	Administration & Operations	Administrator	Durable job queue / worker	Fig 3

4.1.3 Use Case Level 2: Carbon Calculation

Fig 4 refines module 1.4. Preview and commit both include input selection; a commit additionally builds a frozen evidence snapshot, runs the pathway engine (VM0051 for rice_awd, VM0042 with VMD0054 leakage for cropland ALM) and evaluates readiness. Reusing saved signal evidence and superseding a prior version are optional extensions.

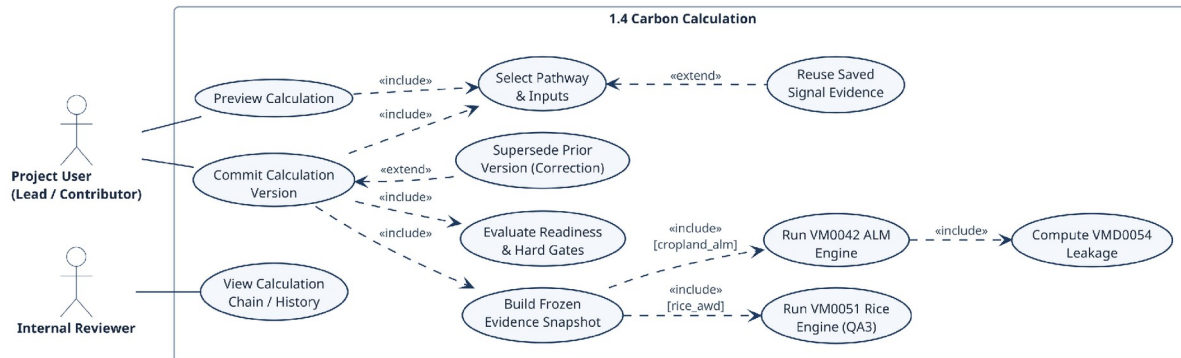


Fig 4: Use Case Level 2 — Carbon Calculation (1.4) with «include» and «extend» relationships.

4.1.4 Use Case Descriptions

Table 5 specifies the actor, precondition, normal flow, alternative/error flow and postcondition of each Level-1 use case.

Table 5: Detailed use-case specifications

Use Case	Actor(s)	Precondition / Trigger	Normal Flow	Alt. / Error Flow	Postcondition
Authenticate & Manage Account	Any user; Administrator	Registered or invited email address	Request OTP or sign in — bcrypt check — JWT stored in httpOnly session — organization and role loaded	Wrong credentials — 401; revoked token version — sign in again; email not	Authenticated session bound to the user's organization

Use Case	Actor(s)	Precondition / Trigger	Normal Flow	Alt. / Error Flow	Postcondition
				configured – OTP unavailable	
Project & Field Management	Project Lead / Contributor	Authenticated organization/project context.	Create/select project – manage members – register field/geometry – assign project/farm membership – update permitted metadata.	Invalid geometry, duplicate/invalid input, or insufficient permission – no write and actionable error.	Authorized project/field context persisted.
Evidence & Season Management	Project User / Internal Reviewer	Field exists; authorized evidence context.	Create/correct season – observations/practice events – attachments – soil/lab/custody – production/livestock evidence.	Validation, permission, storage, or missing-required-data errors remain visible; corrections append history rather than silently replacing prior versions.	Versioned evidence state available to readiness/calculation workflows.
Signal Analytics	Project User / Researcher	rice_and field and monitoring window selected.	Reuse compatible active/cache result where possible – otherwise enqueue signal_run – worker queries Earth Engine – process threshold/ML detector – store result.	Earth Engine, worker, empty-data, detector, or authorization failure – job/error state; no successful result is fabricated.	Completed signal result or explicit failed/cancelled state is available.
Carbon Calculation	Project User	Registered field plus pathway-specific inputs/evidence.	Select pathway and inputs – optionally use saved signal – build snapshot – run VM0051 or VM0042/leakage – evaluate gates/readiness – save version.	Invalid inputs, unsupported/applicability conditions, leakage/SOC hard blocks, or unresolved readiness prevent a ready-for-review state.	Versioned draft or ready-for-review calculation is persisted.
Methodology Readiness	Project User / Internal Reviewer	Active methodology bundle and project/field context.	Resolve bundle/applicability – evaluate registered requirements/corrections – calculate fingerprint/readiness – expose blockers and reviewable items.	Unsupported or automated-only requirements cannot be silently overridden; stale evidence changes the fingerprint/state.	Current readiness and blocking evidence context is displayed/stored.
Internal Review	Project Lead / Internal Reviewer	Specific calculation version is ready_for_review.	Submit calculation – assign reviewer – record review events/findings/comments – resolve or track findings.	Blocked/draft calculation cannot enter normal review; authorization failures are rejected.	Version-pinned internal review trail is retained.
Reporting & Export	Project User / Internal Reviewer	Calculation/history context exists and user is authorized.	Select the relevant verification or calculation record – choose an export format actually available for that record – inspect	Unsupported export, missing context, or legacy provenance limitation is disclosed rather than silently	Supported evidence/report artifact is downloaded with stated provenance

Use Case	Actor(s)	Precondition / Trigger	Normal Flow	Alt. / Error Flow	Postcondition
			history/portfolio context – download.	substituted.	limits.
AI Assistance & Research	Researcher / Authorized User	Authorized project context plus evidence/methodology context or a research dataset.	Build/evaluate AWD or crop models, or request AI explanation – build deterministic packet – provider/job – validate citations/numbers – retain draft.	Missing provider/permission/data, oversized packet, or failed validation – warning/failed job; no authoritative decision is written.	Research metrics or a validated human-review draft is available.
Administration & Operations	Administrator	Administrative authorization.	Manage methodology sources/provider permissions – inspect jobs/workers – retry/cancel supported jobs – use operational/backup tooling.	Permission, stale worker, external-service, or configuration failure remains visible and does not masquerade as success.	Operational configuration and monitoring state are updated or an explicit failure remains.

4.2 Activity Diagrams

The activity diagrams describe the implemented control flow, with swimlanes for the user, the server and the durable worker. Paths that end in a block are drawn as separate branches rather than folded into the main flow. The review flow in Fig 6 relies on one such check: `create_submission()` in `src/projects/reviews.py` accepts a calculation version only if its status is `ready_for_review`, meaning it has no blocking readiness items, and it refuses a second submission for a version that already has one that has not been withdrawn.

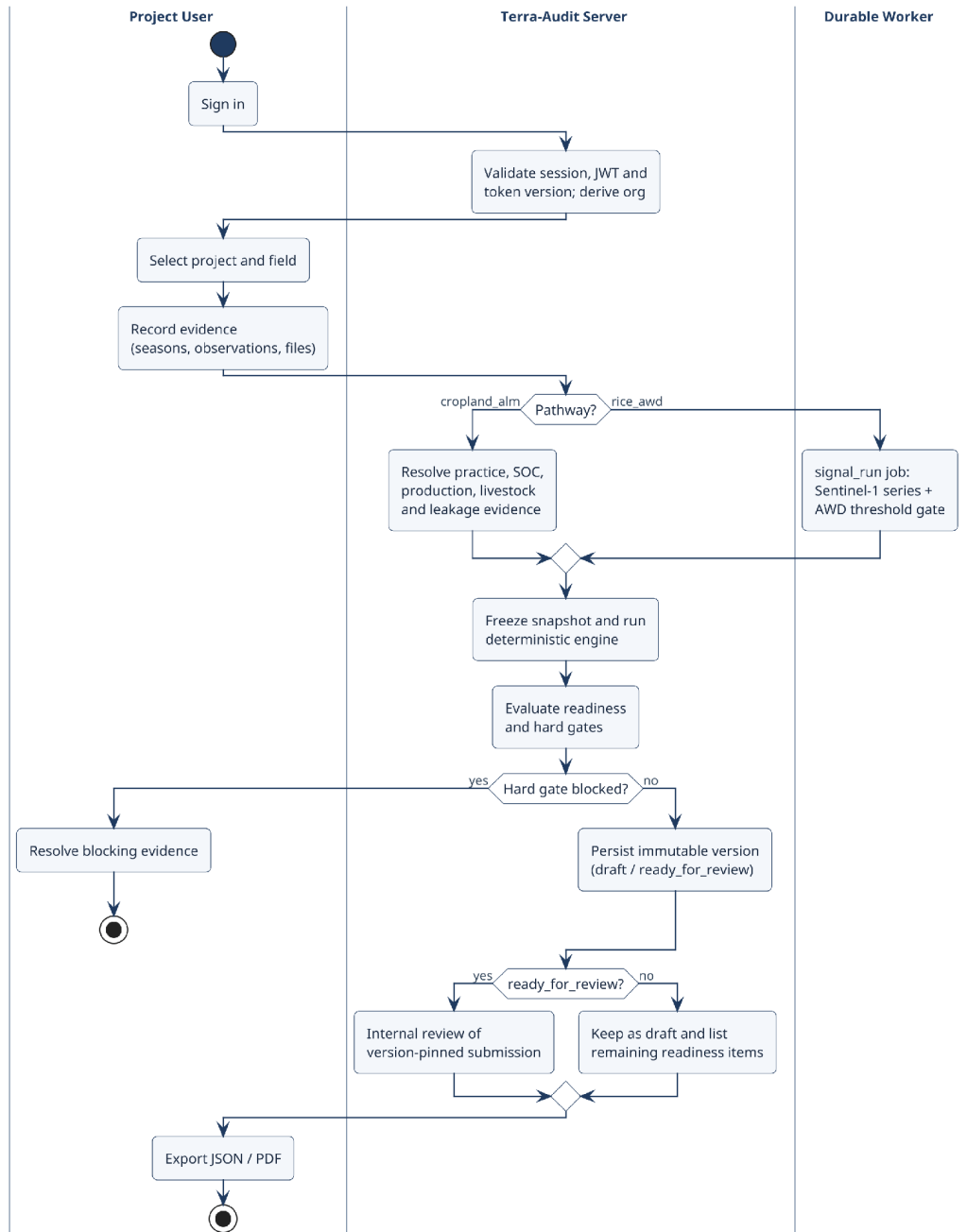


Fig 5: Overall activity diagram with swimlanes for the project user, the Terra-Audit server and the durable worker.

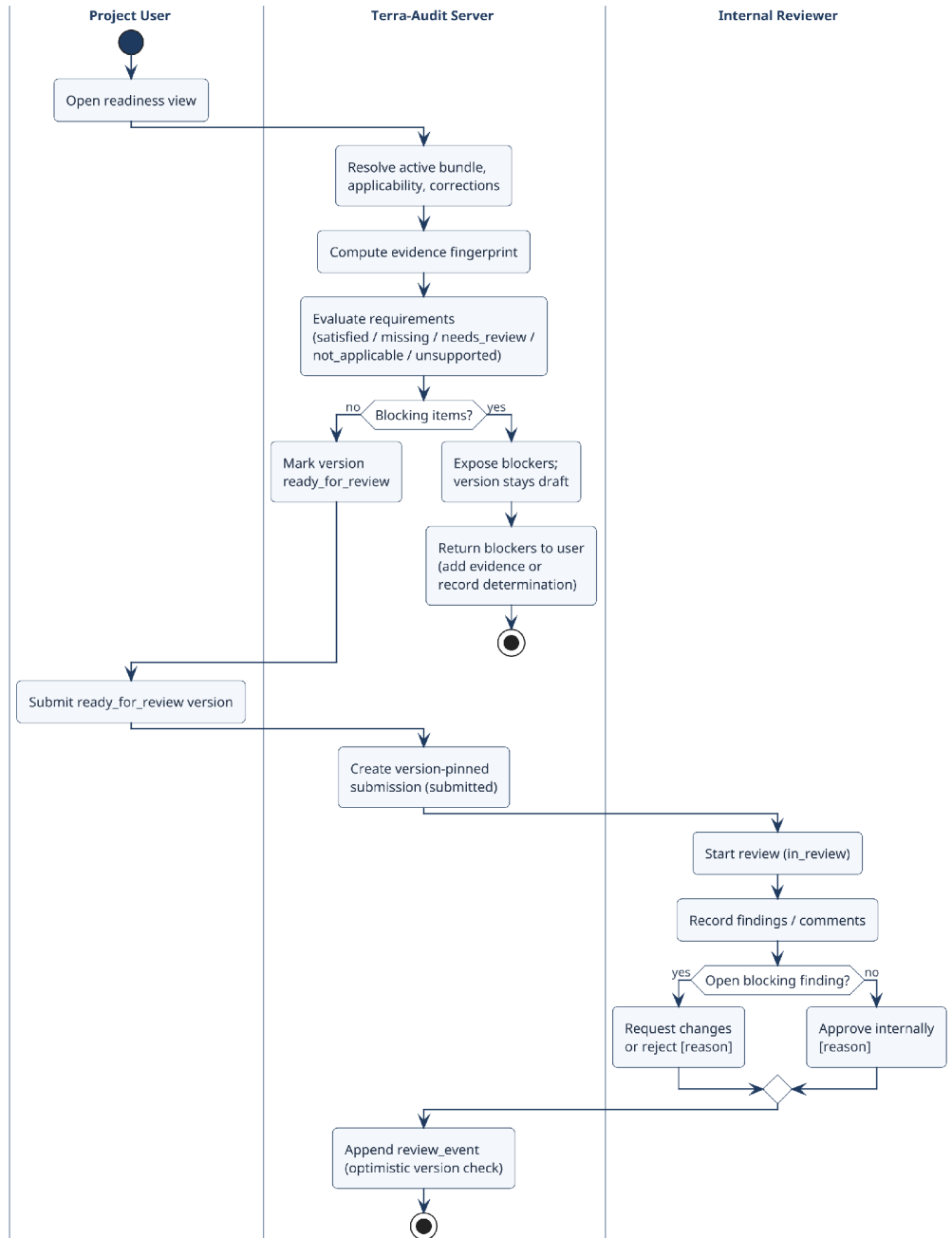


Fig 6: Activity diagram — methodology readiness and internal review with swimlanes.

4.2.1 Cropland ALM Detailed Activity Flow

The ALM flow starts with a `cropland_alm_vm0042` field inside an organization and project, with dated project membership. The user records baseline and project crop seasons, practice schedules, any supported livestock schedules, production records, soil sampling plans, strata and samples, laboratory results, custody events and reviewed SOC mappings (or clearly identified legacy aggregate SOC evidence). A leakage assessment is then saved for the project, bundle and reporting period. When the user commits a calculation, the system freezes the relevant practice, production, leakage and SOC evidence in a snapshot. The restricted VMD0054 v1.1 adapter computes the supported displacement-leakage term, and the VM0042 v2.2 engine computes the SOC and non-SOC terms, uncertainty, allocation and buffer. Readiness and the engine's hard gates run before the version is saved, and only a `ready_for_review` version can enter internal review. The ALM PDF and calculation JSON can then be exported. Several restrictions still apply and stay visible to the user: leakage covers a single field,

mitigation Step 2 is unsupported, nonannual SOC is blocked where annualization is unresolved, and cumulative state across sequential periods is not fully reconciled.

5. Data-Based Modeling

5.1 Entity-Relationship Diagram

Terra-Audit uses a domain-oriented relational schema. Fig 7 shows the core entities in crow's-foot notation with their primary keys and principal reference attributes; Fig 8 details the calculation and internal-review tables. Most tables are tenant-scoped through a composite (org_id, id) primary key. Relationships are logical: the SQLite schema does not declare FOREIGN KEY constraints, so scoped identifiers, application queries and authorization checks enforce referential integrity.

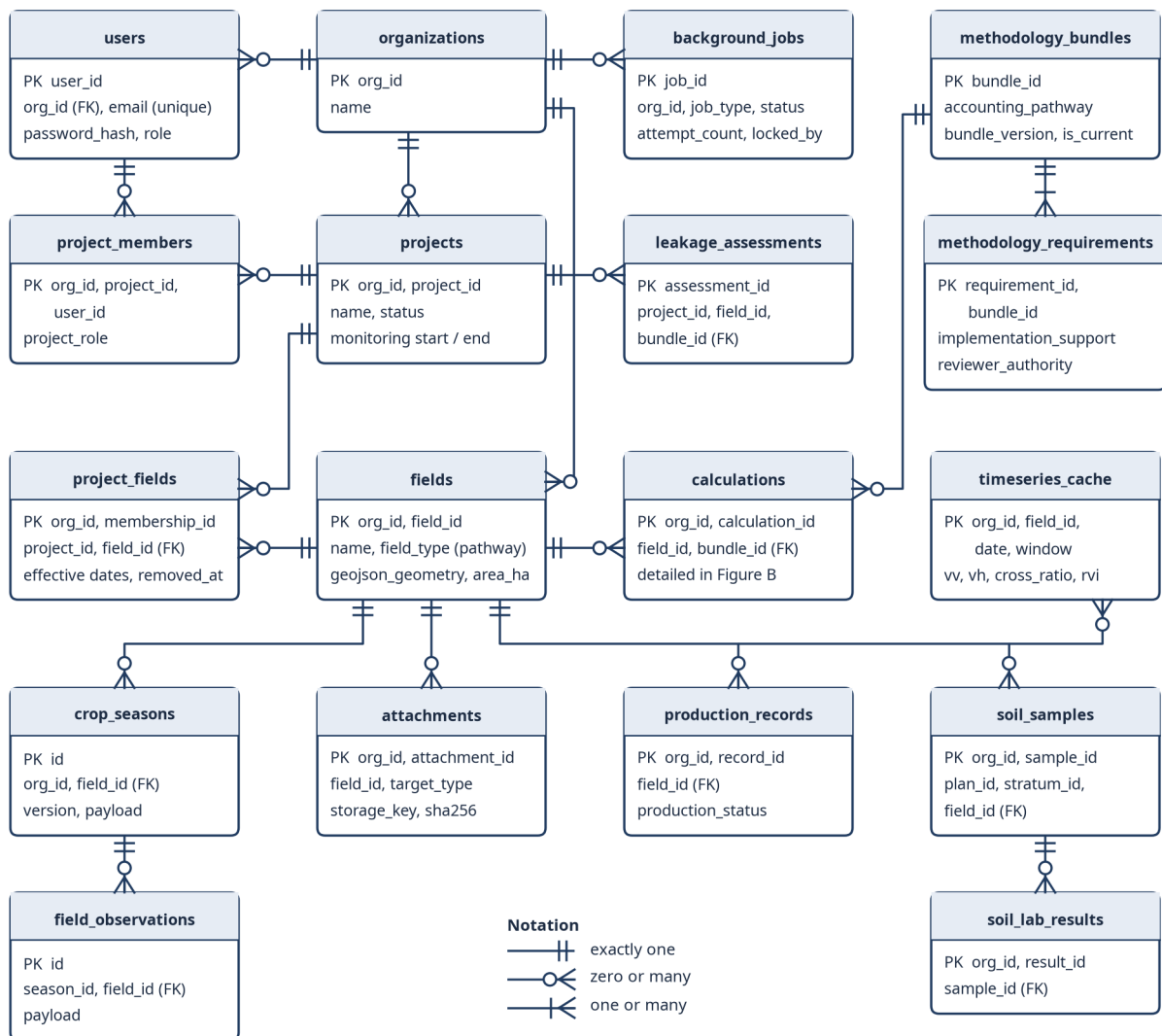


Fig 7: Entity-relationship diagram (crow's-foot notation) of the core persistent entities.

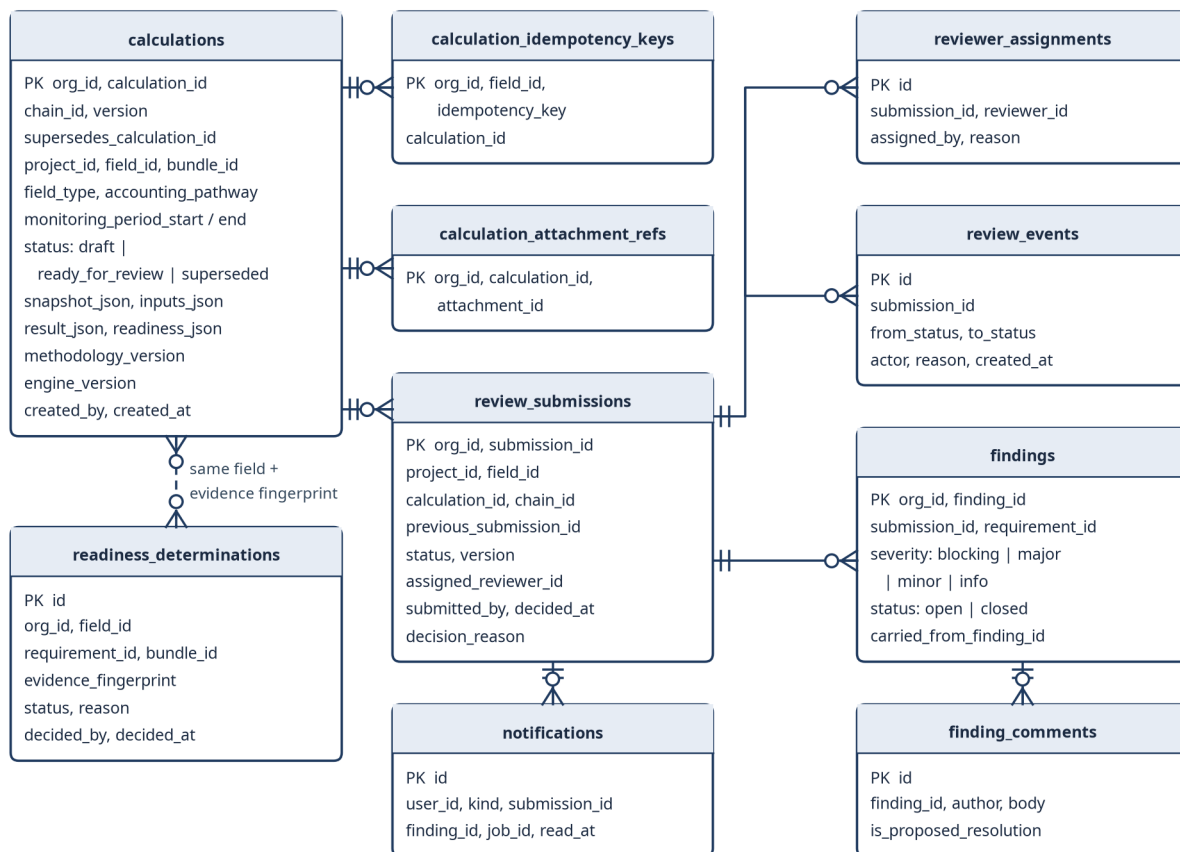


Fig 8: Entity-relationship diagram of the calculation, readiness and internal-review tables.

5.1.1 Relationship Summary

Identity and project context — A user’s access to a project is a ProjectMember record with an explicit role. A field’s membership in a project is an effective-dated ProjectField record rather than a single project_id on the field, so the history of assignments can be kept.

Evidence and sampling — CropSeason, FieldObservation, ObservationReview and PracticeEvent records belong to a field and season and are appended rather than overwritten. Attachments are field-scoped and point to a supported target record. On the ALM side, a soil sampling plan links optional strata, samples, laboratory results, custody events and reviewed SOC mappings; production records and leakage assessments add further field or project evidence.

Calculation provenance — Calculation records are versioned, and the application never edits them in place: chain_id and supersedes_calculation_id record the correction history, and snapshot_json, inputs_json, result_json and readiness_json hold the context frozen at commit time. CalculationAttachmentRef keeps the links to protected attachments. Legacy credit history can still be told apart from snapshot-backed calculations.

Methodology reference data — MethodologyDocument, MethodologyBundle, bundle-document membership and MethodologyRequirement make up the registry shared by all tenants. A tenant’s ProjectMethodologyApplicability record links a project and pathway to a bundle for an effective period. Corrections and their confirmations are stored separately from the global source data.

Internal review — ReviewSubmission pins one immutable calculation version. Reviewer assignments, lifecycle ReviewEvents, Findings and finding comments and resolution activity all belong to this internal workflow, whose states are shown in Fig 19.

5.2 Database Schema

Table 6: Persistent database schema by domain

Domain	Important Entities / Tables	Purpose
Accounts & Identity	organizations, users, pending_registrations, account_links, account_token_versions, account_rate_limits	Tenant identity, account access/recovery, token revocation/versioning, and account-level throttling.
Projects & Land	projects, project_members, project_membership_events, farms, fields, project_fields, farm_fields, quantification_units	Project roles and dated land/field memberships; field records carry the registered accounting pathway and geometry context.
Monitoring & Evidence	crop_seasons, field_observations, observation_reviews, practice_events, monitoring_runs, data_quality_issues, attachments	Versioned crop-season history and structured monitoring/evidence records, including private evidence-file metadata.
Soil / Production / ALM	soil_sampling_plans, soil_strata, soil_samples, soil_lab_results, soil_custody_events, soc_evidence_reviews, alm_practice_schedule, alm_livestock_schedule, production_records, leakage_assessments	Structured ALM evidence for reviewed SOC, practices, production, supported livestock inputs, and restricted leakage assessment.
Calculations & Review	calculations, calculation_idempotency_keys, calculation_attachment_refs, readiness_determinations, review_submissions, reviewer_assignments, review_events, findings, finding_comments	Versioned calculation snapshots/results/readiness plus version-pinned internal review and findings history.
Methodology	methodology_documents, methodology_bundles, bundle-document mappings, methodology_requirements, project_methodology_applicability, methodology_chunks/FTS, correction links/confirmations	Methodology/version/correction registry, retrieval index, applicability context, and requirement definitions used by readiness and AI context.
Jobs / AI / Satellite	background_jobs, job_batches, workers, ai_records, ai_deployments, ai_provider_permissions, timeseries_cache, timeseries_cache_versions; lazy ai_dataset_rows / ai_dataset_versions	Durable asynchronous work, worker state, satellite cache/versioning, provider/model records, and lazily created research-dataset tables.

The SQLAlchemy layer supports SQLite and PostgreSQL. On SQLite, startup runs a series of additive ALTER TABLE statements accumulated over the project, plus a few rebuild-and-copy migrations. PostgreSQL uses the final table definitions with additive changes per module. There is no versioned migration ledger such as Alembic. In the final checks, a fresh SQLite setup, repeated initialization and restart, an upgrade of a representative legacy SQLite database and an SQLite integrity check all passed. PostgreSQL migration on a remote server was not verified, and the existing SQLite-to-PostgreSQL transfer script only handles a legacy subset of the current schema.

5.3 Dataset Description (AI-Assisted Detection)

The classical AWD dataset is built from cached rice time series. Its labels come from the rule-based threshold detector, not from field observations. The features are satellite and time-series values, and the detector's own decision flags are left out. So when Random Forest or XGBoost scores well, it means

the model agrees with the threshold detector; it does not mean the model is accurate against real irrigation events.

The evaluation code requires a field identifier on every row and uses GroupKFold, so no field appears in both a training and a validation fold. Pooled out-of-fold precision, recall, F1, confusion matrices and ROC-style outputs are labelled as agreement with the threshold gate. Saved model bundles also record the satellite processing version and the split strategy, and a bundle built on an older feature version is rejected at load time.

Grouping by field removed the fold-leakage problem of the earlier version. The bigger issue now is where the labels come from: the models are trained to copy the same detector that produced their labels. Sparse drydown events, gaps between Sentinel passes, phenology effects and the lack of measured water levels or irrigation events all limit what these metrics can tell us.

Proper scientific validation would compare detected flood and drydown events with independent data such as water-level tubes, irrigation logs or partner-verified field observations, and report event-level precision and recall, timing error, false drydown rate and results across seasons and locations. The crop models are a separate experiment based on reviewed crop-identity labels. Neither they nor the AWD classifiers can establish AWD compliance under the methodology.

6. Class-Based Modeling

6.1 Analysis Classes

Candidate classes were identified with the grammatical-parse technique [17]: nouns in the usage scenarios (Section 3.6) and the SRS were listed and then filtered by whether they retain information, provide needed services, have multiple attributes and share common operations. Table 7 summarizes the decisions.

Table 7: Analysis-class identification from scenario nouns

Candidate noun(s)	Decision	Resulting design element
User, organization, project member	Accepted	accounts / projects modules; users, project_members
Project, farm, field	Accepted	projects repository; fields, project_fields
Crop season, observation, practice event	Accepted	evidence modules (append-only records)
Soil sample, lab result, production record	Accepted	soil and production evidence modules
Satellite image, backscatter time series	Accepted	SpatialDataEngine; timeseries_cache
Flooding / drydown event, phenology	Accepted	AdaptiveAWDGate
Emission reduction, carbon credit, leakage	Accepted	CarbonAssetEngine, AlmCarbonEngine, VMD0054
Calculation, snapshot, version	Accepted	snapshot / calculations module
Methodology, bundle, requirement	Accepted	methodology registry and readiness
Review, finding, comment	Accepted	reviews module
Job, worker; explanation, provider	Accepted	jobs queue + worker; AI packets / validator
Date, area, location, district	Rejected (attribute)	attributes of field / season
Dashboard, page, button	Rejected (interface)	frontend components only
VVB, Verra registry	Rejected	outside the system boundary

Candidate noun(s)	Decision	Resulting design element
	(external)	

6.2 Class Cards (CRC Model)

CRC cards record each design class's responsibilities and collaborators (Fig 9). Only implementation-backed classes and modules are shown; method-level operations appear in the design class diagram (Fig 10).

SpatialDataEngine Responsibilities Query Sentinel-1 GRD (IW, VV+VH) Derive cross-ratio and RVI bands Median-reduce, Savitzky-Golay smooth Collaborators Google Earth Engine, timeseries_cache	AdaptiveAWDGate Responsibilities Z-score flood detection ($z < -0.8$) Detect drydown events (1.2 sigma) Extract sowing / harvest dates Collaborators SpatialDataEngine, ML predictor	CarbonAssetEngine Responsibilities VM0051 QA3 CH4 scaling factors N2O irrigation penalty, uncertainty Block results above 60,000 tCO2e/yr Collaborators AdaptiveAWDGate, snapshot
AlmCarbonEngine Responsibilities VM0042 SOC stock change, ER / CR Non-SOC, livestock, buffer terms Block missing leakage evidence Collaborators VMD0054 leakage, AlmPracticeValidator	Snapshot & Calculations Responsibilities Freeze evidence / inputs / results Idempotent, versioned commit Supersede prior version on correction Collaborators Carbon engines, readiness	Readiness Responsibilities Resolve bundle and applicability Evaluate requirement statuses Compute evidence fingerprint Collaborators Methodology registry, corrections
Reviews Responsibilities Version-pinned submissions Guarded state transitions Findings, comments, notifications Collaborators calculations, users	Job Queue & Worker Responsibilities Persist and claim jobs atomically Heartbeat, lease, retry, cancel Dispatch typed job handlers Collaborators Signal / AI / ingest handlers	AI Packet & Validator Responsibilities Build bundle-scoped context packet Run 14 server-side output checks Store draft for human review Collaborators AI providers, methodology library

Fig 9: CRC cards for the nine core design classes.

6.3 Design Class Diagram

The code is modular, but it does not follow a strict class/service/repository pattern. Responsibilities are split across domain packages, with classes where they help and plain module functions elsewhere. The main units are: accounts and authentication (src/accounts, backend/security.py); project and internal-review repositories (src/projects); evidence modules for seasons, monitoring, soil and production; the methodology registry, library and readiness modules; the rice and ALM carbon engines; the leakage adapter; snapshot and calculation-version storage; signal acquisition and processing; the job queue and worker handlers; AI packets, providers, validators and ML models; reporting; and persistence and storage.

The UI, background jobs, satellite models and language models can supply inputs, analysis or text, but accounting values, readiness states and validation results are decided only by server code. A probabilistic component therefore cannot change the methodology logic unnoticed.

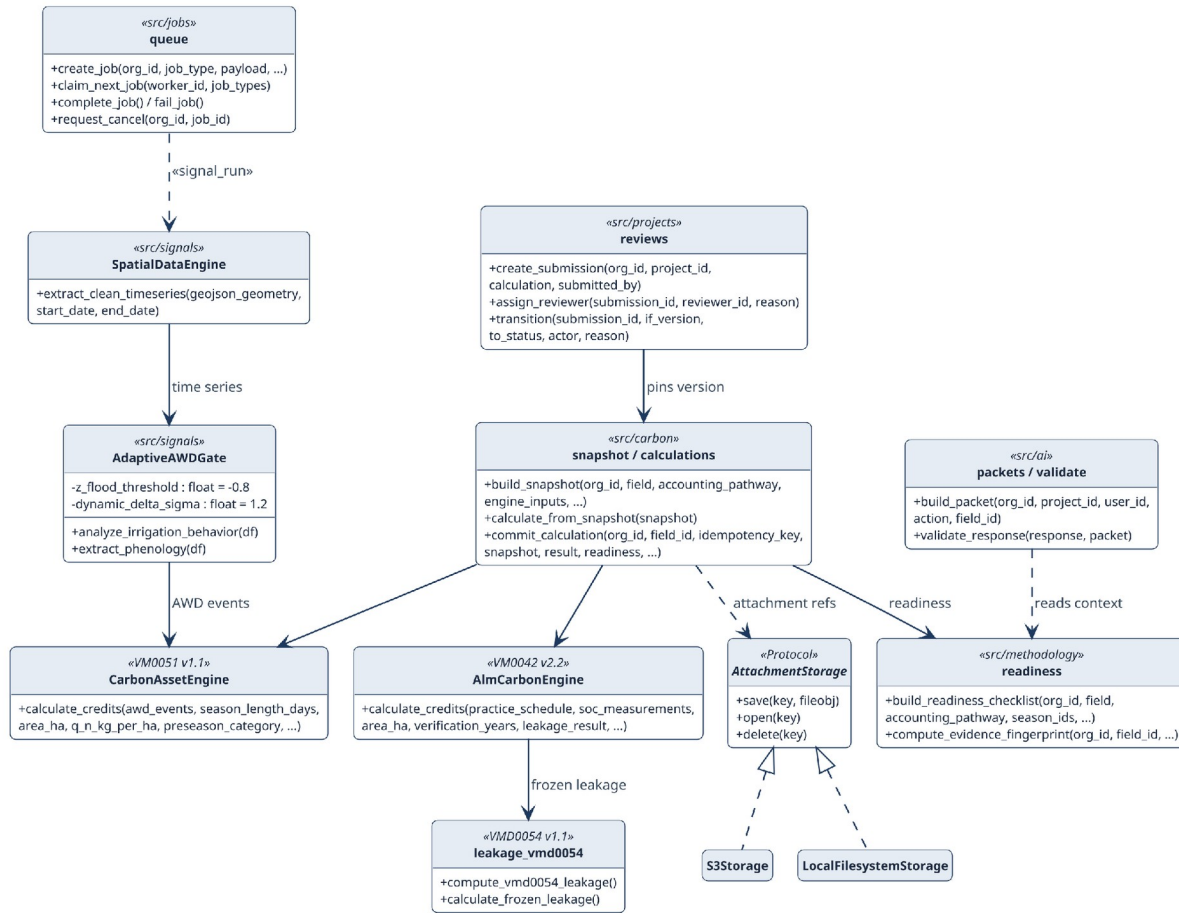


Fig 10: Design class diagram: principal classes and modules, key operations and dependencies.

6.3.1 Design Class to Implementation Mapping

Since the code is organized by capability and not as a single object hierarchy, Table 8 names the actual classes and module-level operations that carry each design responsibility.

Table 8: Design classes and their implementing modules

Design class	Module	Key operations	Responsibility
CarbonAssetEngine	src/carbon/rice.py	calculate_credits()	Scoped VM0051 v1.1 QA3 rice result, intermediates, hard gates
AlmCarbonEngine	src/carbon/alm.py	calculate_credits()	Scoped VM0042 v2.2 SOC/non-SOC terms, uncertainty, buffer, blocking flags
SpatialDataEngine	src/signals/earth_engine.py	extract_clean_timeseries()	Sentinel-1 VV/VH, cross-ratio and RVI series
AdaptiveAWDGate	src/signals/threshold_gate.py	analyze_irrigation_behavior(), extract_phenology()	Flood/drydown and sowing/harvest indicators
Snapshot & versioning	src/carbon/snapshot.py, calculations.py	build_snapshot(), commit_calculation()	Frozen context; idempotent immutable versions
Readiness	src/methodology/readiness.py	build_readiness_checklist()	Requirement statuses and bundle context
Internal review	src/projects/reviews.py	create_submission(), transition()	Version-pinned, guarded review transitions
Durable jobs	src/jobs/queue.py	create_job(), claim_next_job(), fail_job()	Persisted job lifecycle, lease, retry, cancel

Design class	Module	Key operations	Responsibility
AttachmentStorage	src/persistence/storage.py	save(), open(), delete()	Private local or S3-compatible artifacts
AI packet & validator	src/ai/packets.py, validate.py	build_packet(), validate_response()	Bounded context; server-side output checks
Reporting	src/reporting/reports.py	report generators	Legacy and snapshot-backed exports

Browser requests take a separate route. frontend/lib/api.ts calls the Next.js catch-all API proxy, which reads the httpOnly session on the server and forwards an authenticated request to FastAPI. Browser state is never trusted for accounting or authorization.

7. Architectural Design

7.1 Architectural Context Diagram

The architectural context diagram (ACD) [17] in Fig 11 shows the systems around Terra-Audit. Superordinate systems use its output: VVB validation/verification and the Verra registry receive exported evidence packages but are not part of the system. Subordinate systems are the ones Terra-Audit depends on: Google Earth Engine, the AI providers, the Brevo email API, the relational database and object storage. Peers exchange data at the same level; here these are farm production spreadsheets imported as CSV and Verra methodology documents loaded into the methodology library. The actors are the human users.

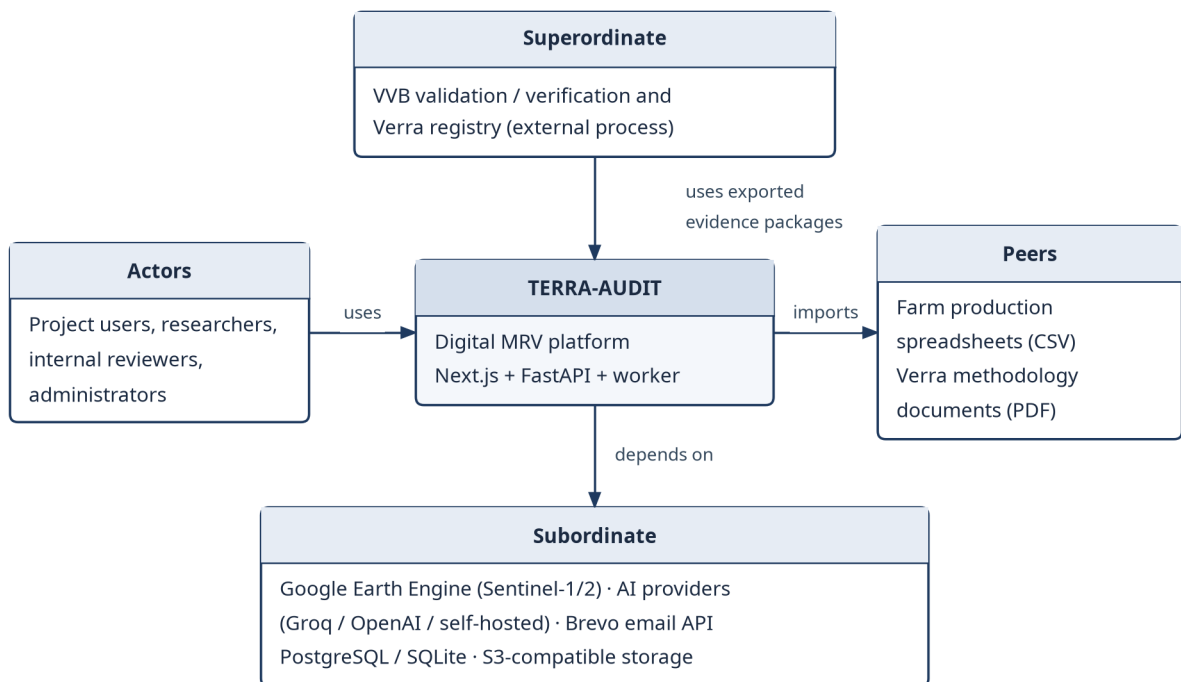


Fig 11: Architectural context diagram (ACD) of Terra-Audit.

7.2 Defining Archetypes

Two archetypes shape the system. The application as a whole is layered (Fig 12): presentation, application/API, domain and infrastructure, with each layer calling only the one below it, and the accounting logic sits in the domain layer. The rice signal path is a pipe-and-filter pipeline (Fig 13). Each filter transforms the Sentinel-1 series (smoothing, for example, uses a Savitzky–Golay filter [10])

and hands it to the next one, which makes every stage easy to test on its own; on a cache hit the acquisition stage is skipped.

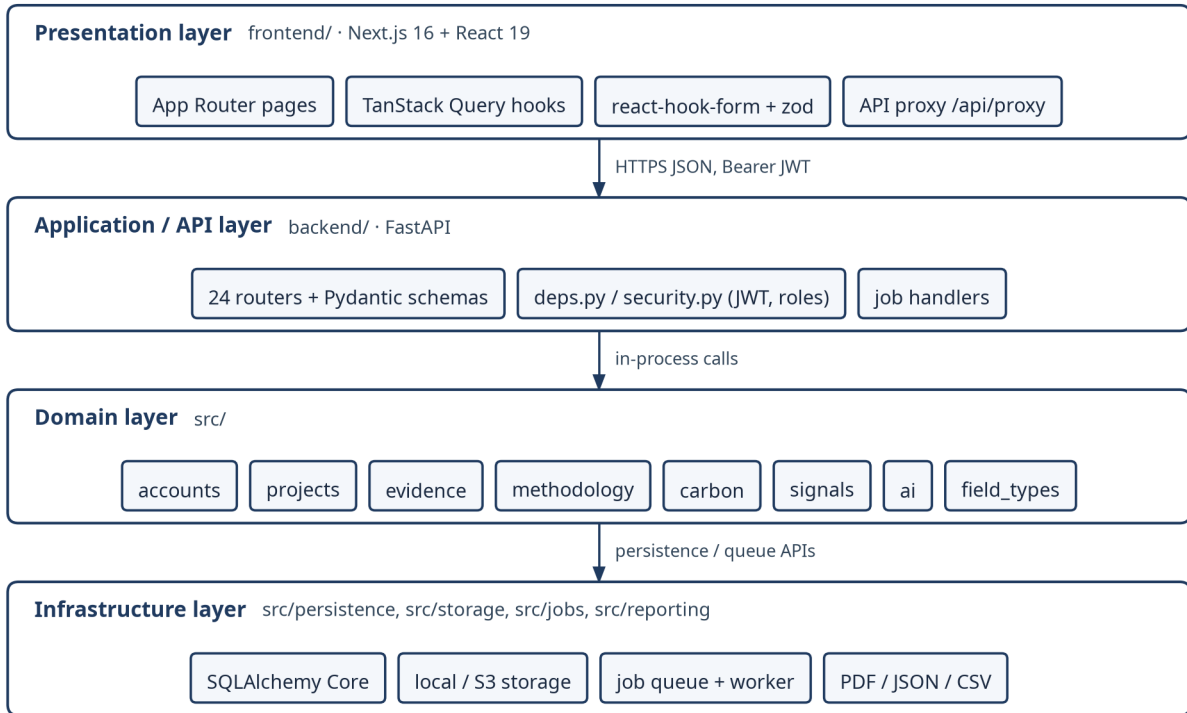


Fig 12: Layered architecture archetype instantiated for Terra-Audit.

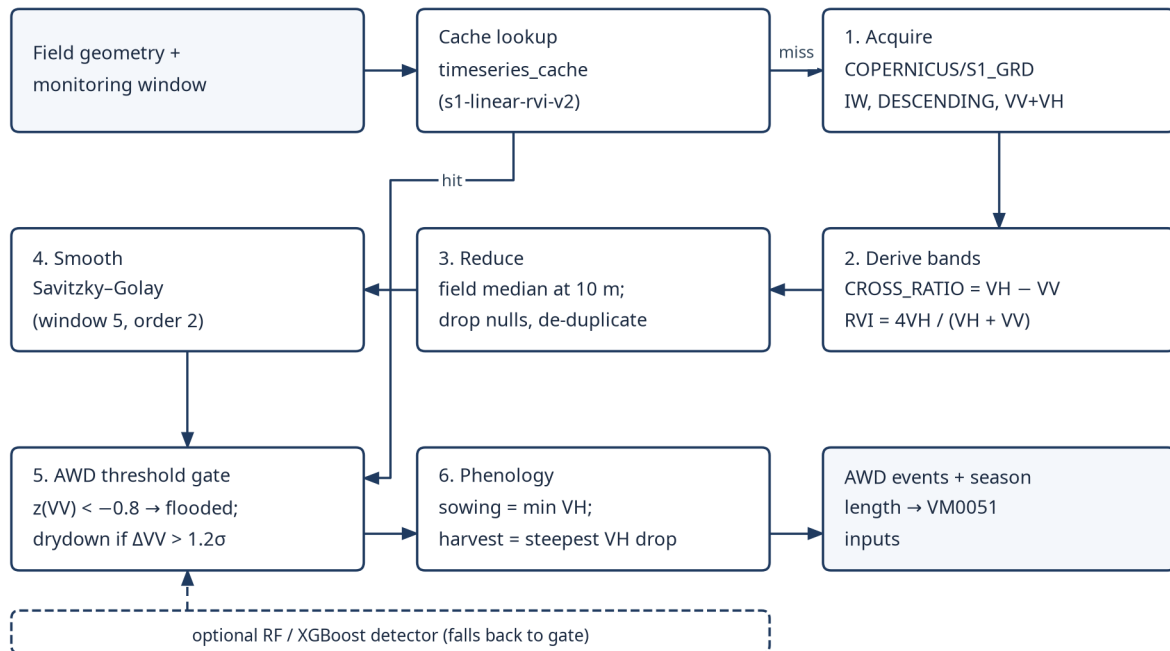


Fig 13: Pipe-and-filter archetype: the Sentinel-1 signal-processing pipeline of the rice AWD pathway.

7.3 Top-Level Components

Fig 14 shows the top-level components that come out of these archetypes. The Next.js frontend talks to FastAPI through an authenticated server-side proxy. FastAPI routers call shared Python domain modules. Long-running signal and AI tasks are written to a queue in the database and picked up by a separate worker. Table 9 lists the responsibility of each component.

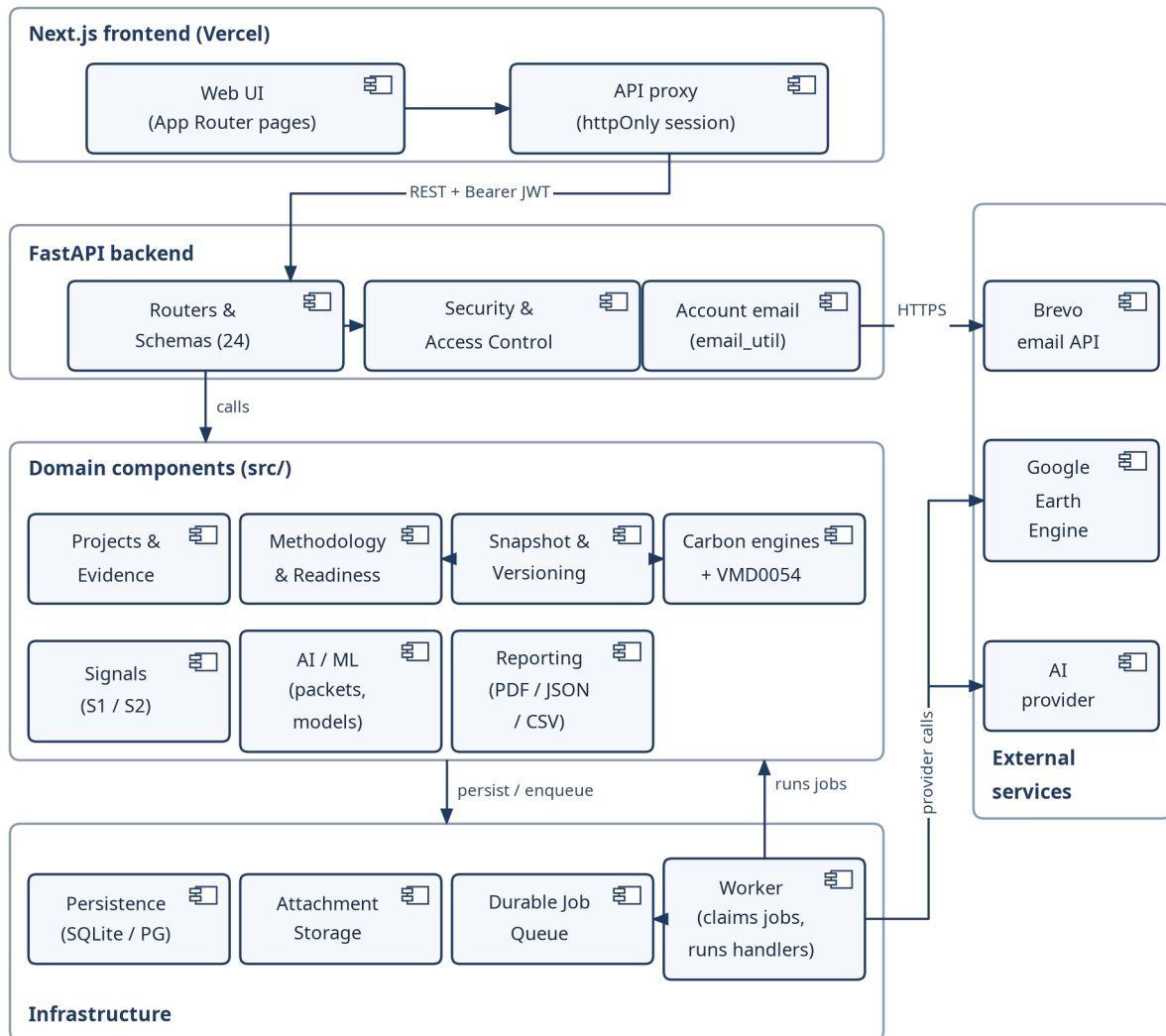


Fig 14: Top-level component diagram of Terra-Audit.

Table 9: Component responsibilities

Component	Responsibility
Next.js Frontend & API Proxy	Renders authenticated pages/components, manages client state/queries, and forwards browser requests through the server-side authenticated API proxy rather than exposing bearer tokens directly to client code.
FastAPI Routers & Schemas	Own HTTP orchestration, request/response validation, route composition, and handoff to shared domain packages.
Accounts / Security / Authorization	Implements identity, password/account-recovery flows, JWT validation, token-version checks, organization scoping, and project/admin access gates.
Projects & Evidence	Manages projects, farms, fields, memberships, crop seasons/corrections, observations, attachments, soil/SOC, production, and internal-review domain records.
Methodology & Readiness	Maintains methodology documents/bundles/corrections, project applicability, requirement definitions, retrieval chunks, evidence fingerprints, and deterministic readiness states.
Carbon Accounting & Leakage	Runs scoped VM0051 v1.1 rice and VM0042 v2.2 ALM calculations plus the restricted VMD0054 v1.1 leakage adapter; AI does not own accounting values.
Snapshots & Calculation Versioning	Builds frozen calculation context, stores inputs/results/readiness, manages idempotency and supersession/version chains, and preserves historical numerical

Component	Responsibility
	payloads under normal application updates.
Signals / Earth Engine	Queries and processes Sentinel-1 observations, maintains compatible cache versions, applies threshold/phenology logic, and supports selected experimental ML detectors.
Durable Jobs & Worker	Persists long-running work in the database-backed queue; separate worker processes execute signal, AI, methodology-ingestion and related jobs with heartbeat/retry/cancellation state.
AI / ML / Providers / Validation	Supports classical AWD ML, managed crop models, deterministic AI context packets, provider adapters, structured-response validation, and human-review draft explanations.
Reporting & Export	Generates legacy verification PDF/JSON/CSV artifacts, versioned calculation JSON, snapshot-backed ALM calculation PDF, and history/portfolio outputs, with different provenance guarantees for snapshot-backed and legacy paths.
Persistence & Storage	Provides SQLAlchemy-backed SQLite/PostgreSQL access plus local/private S3-compatible artifact storage; application logic enforces relationships because the verified SQLite schema declares no foreign keys.

7.4 Mapping Requirements to Software Architecture

Table 10: Mapping functional requirements to architectural elements

Requirement(s)	Architectural element	Layer
FR1, FR17	Accounts; Security & Access Control (backend/security.py, deps.py)	Application / Domain
FR2, FR5, FR6	Projects & Evidence (src/projects, src/evidence)	Domain
FR3, FR4	Signals pipeline + Durable Job Queue + Worker	Domain / Infrastructure
FR7, FR8	Carbon Engines + VMD0054 leakage adapter	Domain
FR9, FR10	AI / ML (src/ai) executed through ai_explain / ai_train jobs	Domain / Infrastructure
FR11, FR12	Reporting; calculation history and portfolio views	Domain / Presentation
FR13	Methodology & Readiness	Domain
FR14	Snapshot & Versioning	Domain
FR15	Internal review (src/projects/reviews.py)	Domain
FR16	Operations views (queue status, product readiness)	Presentation / Application
FR18	Durable Job Queue + Worker	Infrastructure

7.5 Technology Stack

Table 11: Technology stack

Layer	Technology	Purpose
Frontend	Next.js 16.3, React 19.2, TypeScript 5	App Router pages and server-side API proxy
Frontend	TanStack Query 5, react-hook-form 7 + zod 4, Tailwind CSS 4	Server state, validated forms, styling
Frontend	Leaflet 1.9 / react-leaflet 5, Recharts 3, KaTeX	Field maps and drawing, charts, equations
Backend	Python 3.12, FastAPI, Uvicorn, Pydantic	REST API and request validation
Backend	SQLAlchemy 2 (Core), SQLite / PostgreSQL (psycopg2)	Persistence

Layer	Technology	Purpose
Backend	PyJWT (HS256), bcrypt	Authentication and password hashing
Analytics	earthengine-api, pandas, NumPy, SciPy, Shapely	Sentinel-1/2 acquisition, smoothing, geometry
ML / AI	scikit-learn, XGBoost, joblib; httpx to Groq / OpenAI / self-hosted LLMs	AWD and crop models; explanations
Reporting / storage	fpdf2, pypdf, boto3	PDF export, methodology ingest, S3 storage
DevOps	Docker, docker-compose, GitHub Actions, pytest, node --test, ESLint	Build, CI and testing

7.6 Design Evolution and Alternatives

The architecture changed a good deal after the midterm prototype. Streamlit was replaced with Next.js and FastAPI, mainly so that authentication, multi-user permissions, API contracts and interface states could be kept apart from the domain code. Signal and AI tasks used to run inside the web request; they now go through a database-backed queue and a separate worker. The old calculation model stored only a current value, and it was replaced by versioned snapshots with readiness attached so that earlier results can still be audited. Carbon equations and methodology rules stay in ordinary code, and the LLM is limited to writing draft explanations. SQLite is still used for local work and tests, with PostgreSQL and private object storage for hosting. All of this made the system more complex to build, but it fits an audit workflow far better than the single-user prototype did.

Before the durable queue was added, signal and AI work ran inside the API process, so a job's life was tied to that process. Now each job is a row in the `background_jobs` table with a status, an attempt counter (three attempts by default) and a 300-second lease. A worker claims a job with a conditional UPDATE that succeeds only while the row is still pending, so two workers cannot take the same job, and the same SQL works on SQLite and PostgreSQL. If a worker dies, its heartbeat stops and the job is reclaimed once the lease expires. Errors that retrying cannot fix are raised as `InvalidJobRequest` and fail at once; `test_insufficient_benchmark_returns_actionable_error_without_retrying` checks that a crop benchmark with too few eligible examples ends with `attempt_count` 1 and `error_kind` 'invalid' instead of using up all its retries. The price is a second process to deploy and monitor, and jobs are not picked up instantly because workers poll the table. A broker such as Celery or Redis would have added another service to run, which this project's scale does not justify, and callers only use `create_job()` and `claim_next_job()`, so the backend could be replaced later.

8. Component-Level Design

8.1 Persistent Data Sources and Interfaces

All transactional data goes through SQLAlchemy, with SQLite for local work and tests and PostgreSQL for hosted deployments. Evidence attachments and managed-model files are kept private, either on the local filesystem or in an S3-compatible bucket. Several other kinds of data live in the same database: satellite observations in versioned time-series cache tables, methodology source documents split into document and chunk records, and the background jobs and worker heartbeats that make up the queue. The frontend never reads these stores directly; it goes through the application modules.

8.1.1 Context-Level Data Flow (DFD Level 0)

The Level-0 diagram treats Terra-Audit as a single process and shows only the external actors and services and the main information they exchange with it. Data stores and internal processes are left for Level 1.

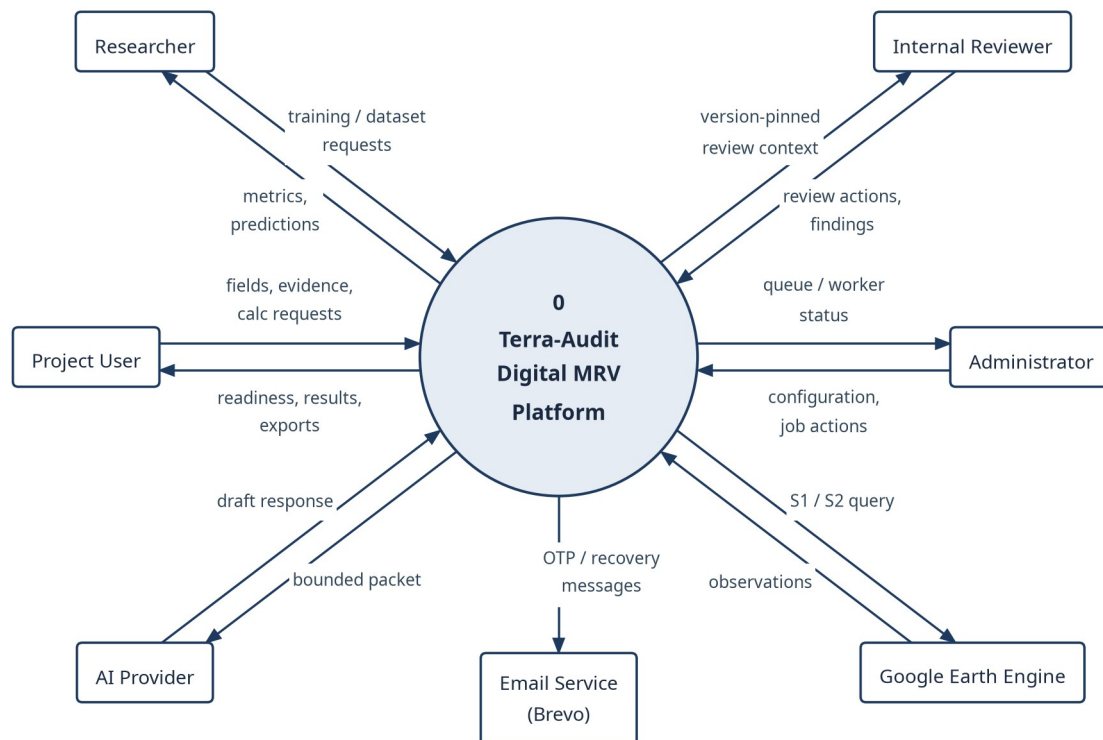


Fig 15: Level-0 data-flow diagram: external entities and the information they exchange with Terra-Audit.

8.1.2 Level-1 Data Flow (DFD Level 1)

Level 1 splits that single process into five processes, each matching a part of the implementation, and adds five logical data stores. The external flows are the same as in Fig 15.

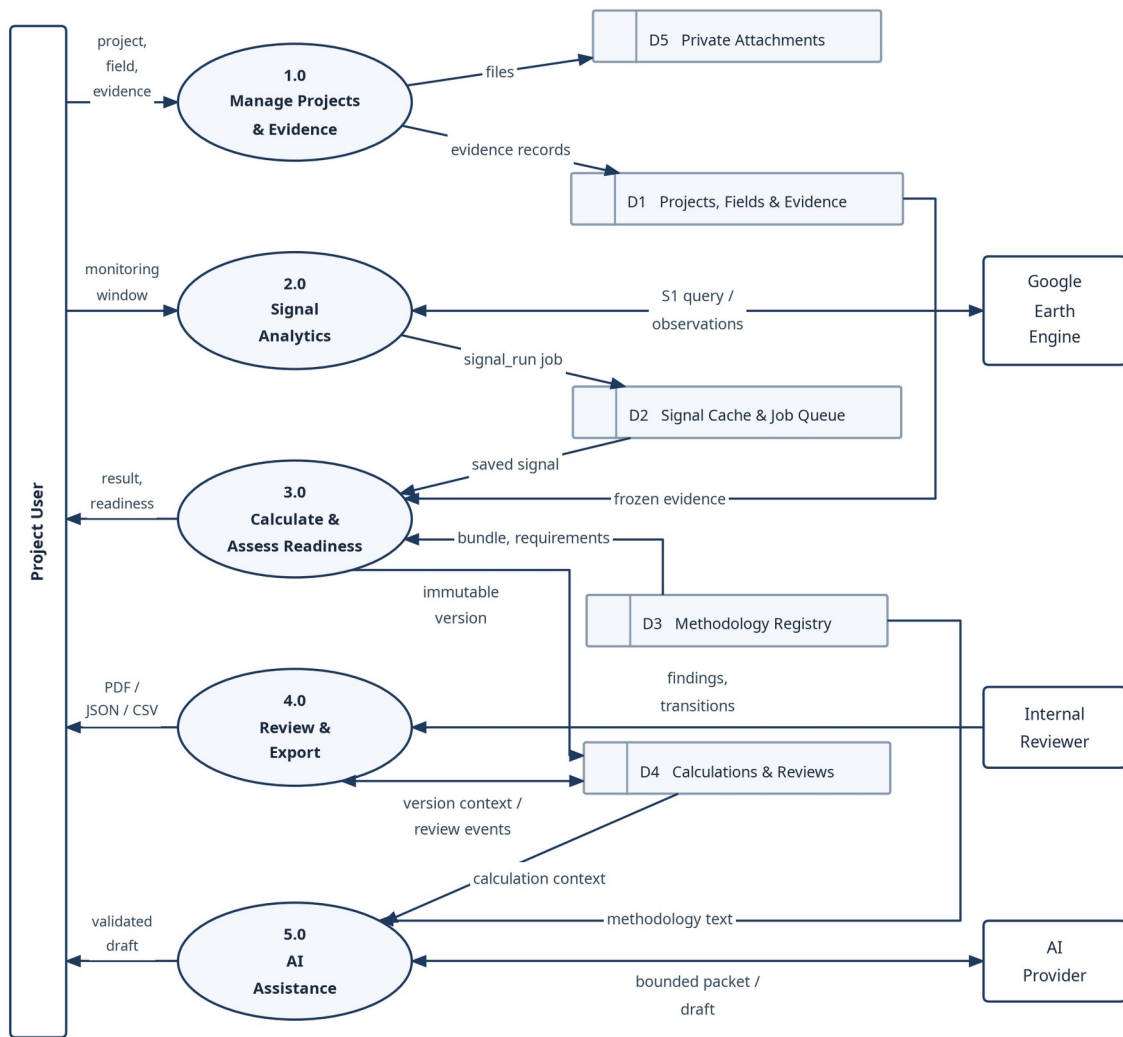


Fig 16: Level-1 data-flow diagram: processes, data stores and external flows.

8.2 API Design

The backend exposes 24 FastAPI routers. Apart from the public sign-in, registration and recovery routes under `/auth` and the `/health` check, every route requires a bearer JWT, and the organization is derived from the token, never from the request. Table 12 groups the representative endpoints.

Table 12: REST API summary by router group

Router group	Representative endpoints	Purpose
Auth & accounts	POST <code>/auth/login</code> , GET <code>/auth/me</code> , POST <code>/auth/register/request-otp</code> , <code>/auth/recovery/*</code>	Sign-in, OTP registration, recovery
Team	GET/POST <code>/team/users</code> , POST <code>/team/invitations</code>	Organization users and invitations
Fields & farms	GET/POST <code>/fields</code> , PATCH <code>/fields/{id}</code> , POST <code>/fields/parse/{geojson kml upload coordinates}</code>	Field registration and geometry
Projects	<code>/projects</code> , <code>/projects/{id}/members</code> , <code>/projects/{id}/fields</code>	Projects, roles, dated memberships
Evidence	<code>/fields/{id}/crop-seasons/...</code> , <code>/attachments</code> , <code>/fields/{id}/soil-*</code> , <code>/production-records</code>	Seasons, observations, files, soil and production
Signals &	POST <code>/fields/{id}/signal-runs</code> , GET <code>/signal-runs/{job_id}</code> ,	Sentinel analytics and batches

Router group	Representative endpoints	Purpose
monitoring	/projects/{id}/monitoring/bulk-run	
Calculations	POST .../calculations/preview, POST /fields/{id}/calculations, GET /calculations/{id}/chain	Preview, commit, history
Methodology	/methodology/bundles, /projects/{id}/methodology-applicability, /methodology/corrections	Registry, applicability, corrections
Reviews	/projects/{id}/submissions, /submissions/{id}/transition, /submissions/{id}/findings	Internal review workflow
AI	POST /projects/{id}/ai/explain, POST /ai/train, POST /ai/dataset/build	Explanations and ML
Export	GET /calculations/{id}/evidence/pdf, /fields/{id}/verifications/{vid}/evidence/{pdf json csv}	Evidence packages
Operations	GET /admin/queue-status, /admin/product-readiness, /portfolio/summary, /health	Monitoring and health

8.3 Behavioral Design

Three views describe the system's behavior: an event table (Table 13), the persisted state machines (Fig 17–Fig 19) and sequence diagrams of the three highest-risk workflows (Fig 20–Fig 22).

8.3.1 Behavioral Event Table

Table 13: Behavioral event table

Event	Trigger / precondition	System response	Resulting state
Signal run requested	Project user; rice_awd field and window	Return compatible cache or enqueue a deduplicated signal_run; worker fetches Sentinel-1 and stores the result	pending → running → done / error / cancelled
Calculation committed	Writer role; valid inputs; new Idempotency-Key	Freeze snapshot, run engine, evaluate readiness, persist version	draft or ready_for_review; prior version superseded on correction
Review submitted	Calculation is ready_for_review	Create version-pinned submission and review event	submitted
Review action	Assigned reviewer; current if_version	Record finding/comment or allowed transition; open blocking finding prevents approval	in_review / changes_requested / internally_approved / rejected / withdrawn
Data-quality issue	Monitoring run detects a problem	Upsert open issue; acknowledge or resolve with actor and reason	open → acknowledged → resolved
AI explanation requested	Provider permission; allowed action	Return cached validated draft or enqueue ai_explain	pending → running → done / error

8.3.2 State Diagrams

Fig 17 shows the durable job lifecycle, including reclaim after a lease expires and the retry limit. Fig 18(a) shows that a calculation version is never edited in place: a correction creates version N+1, which supersedes the previous one; Fig 18(b) shows the data-quality issue lifecycle. Fig 19 shows the internal review transitions enforced by the server.

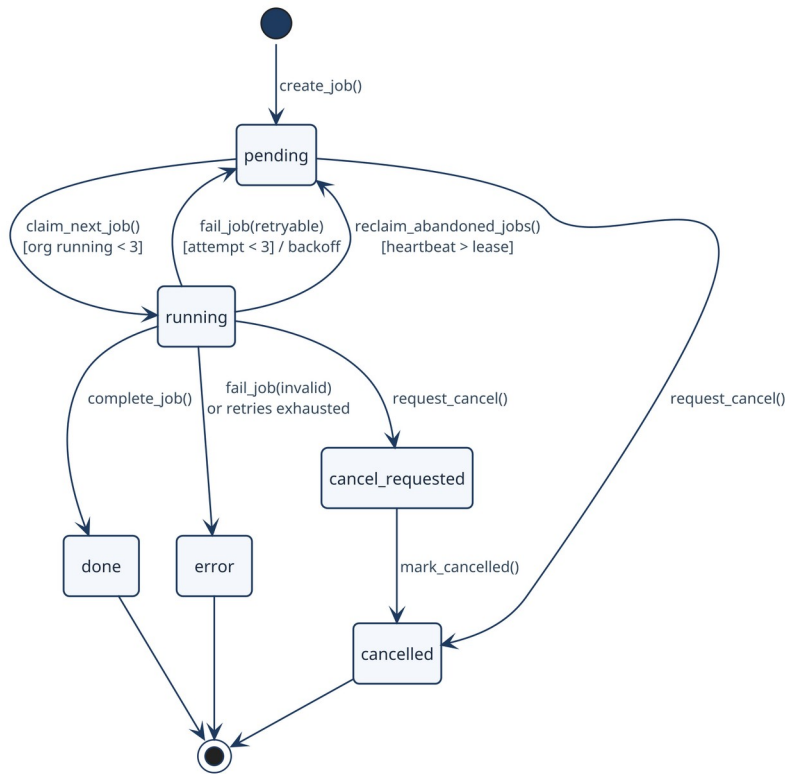


Fig 17: State diagram — durable background job lifecycle (src/jobs/queue.py).

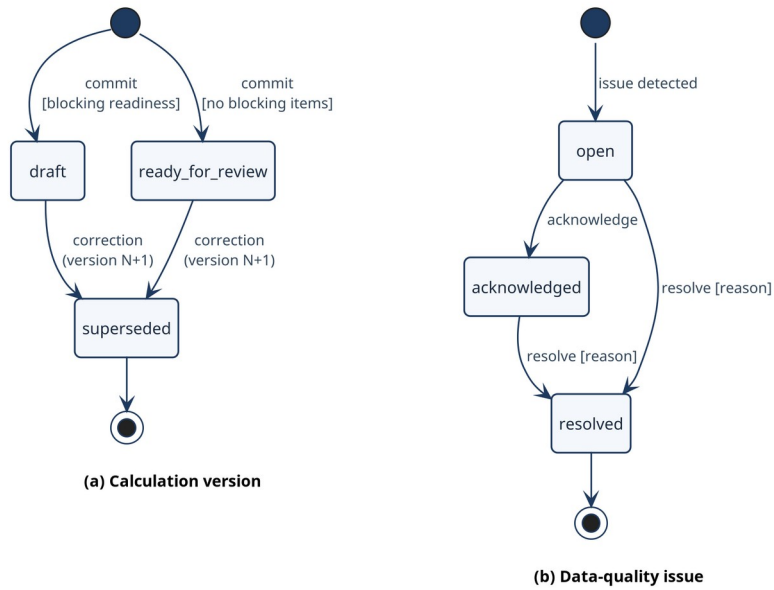


Fig 18: State diagrams — (a) calculation version lifecycle and (b) data-quality issue lifecycle.

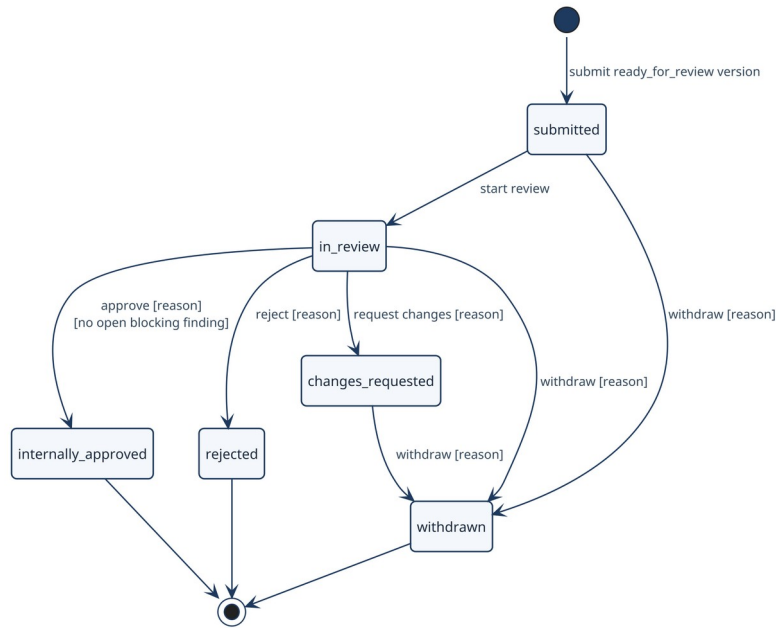


Fig 19: State diagram — internal review submission lifecycle (src/projects/reviews.py).

8.3.3 Sequence Diagrams

The three sequence diagrams cover committing an immutable calculation, running a durable signal job and generating an AI explanation. Internal review is not repeated here, since the readiness/review activity diagram (Fig 6) and the review state machine (Fig 19) already show it. The lifelines are the actual Next.js, FastAPI, domain and worker boundaries in the code.

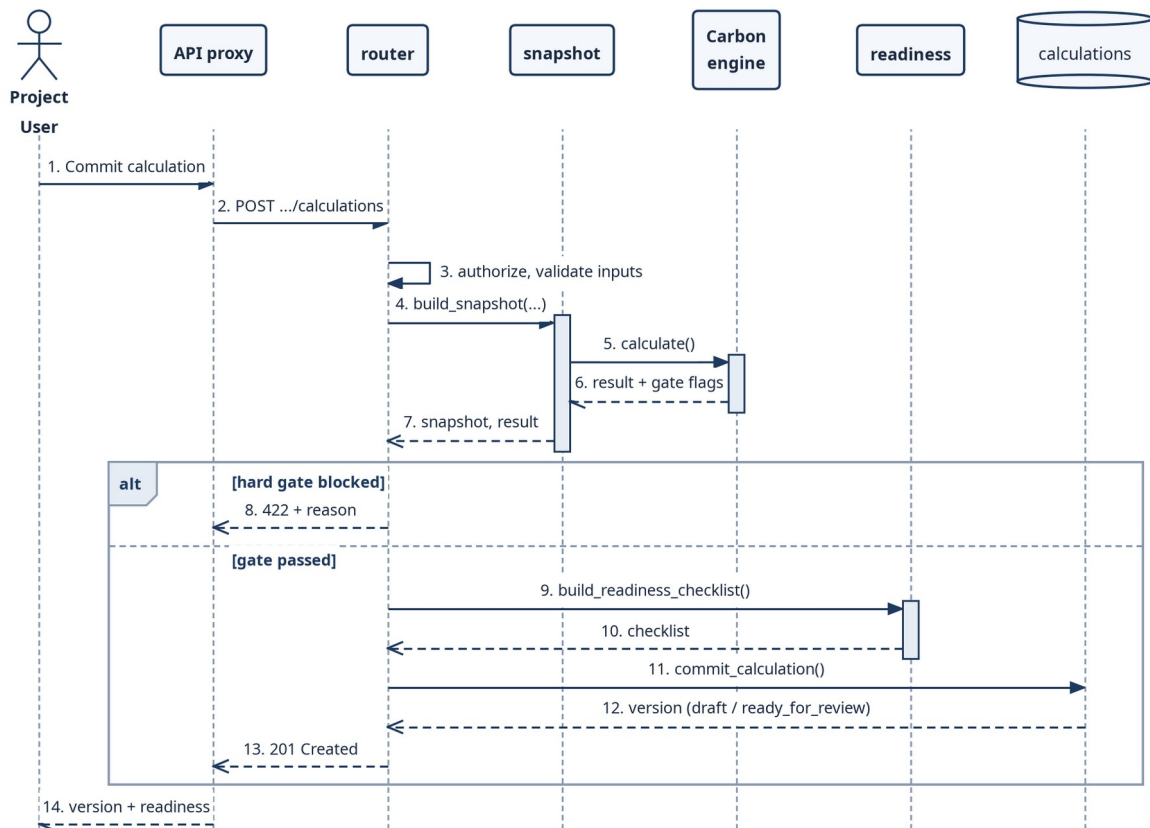


Fig 20: Sequence diagram — calculation commit from authenticated request to immutable version.

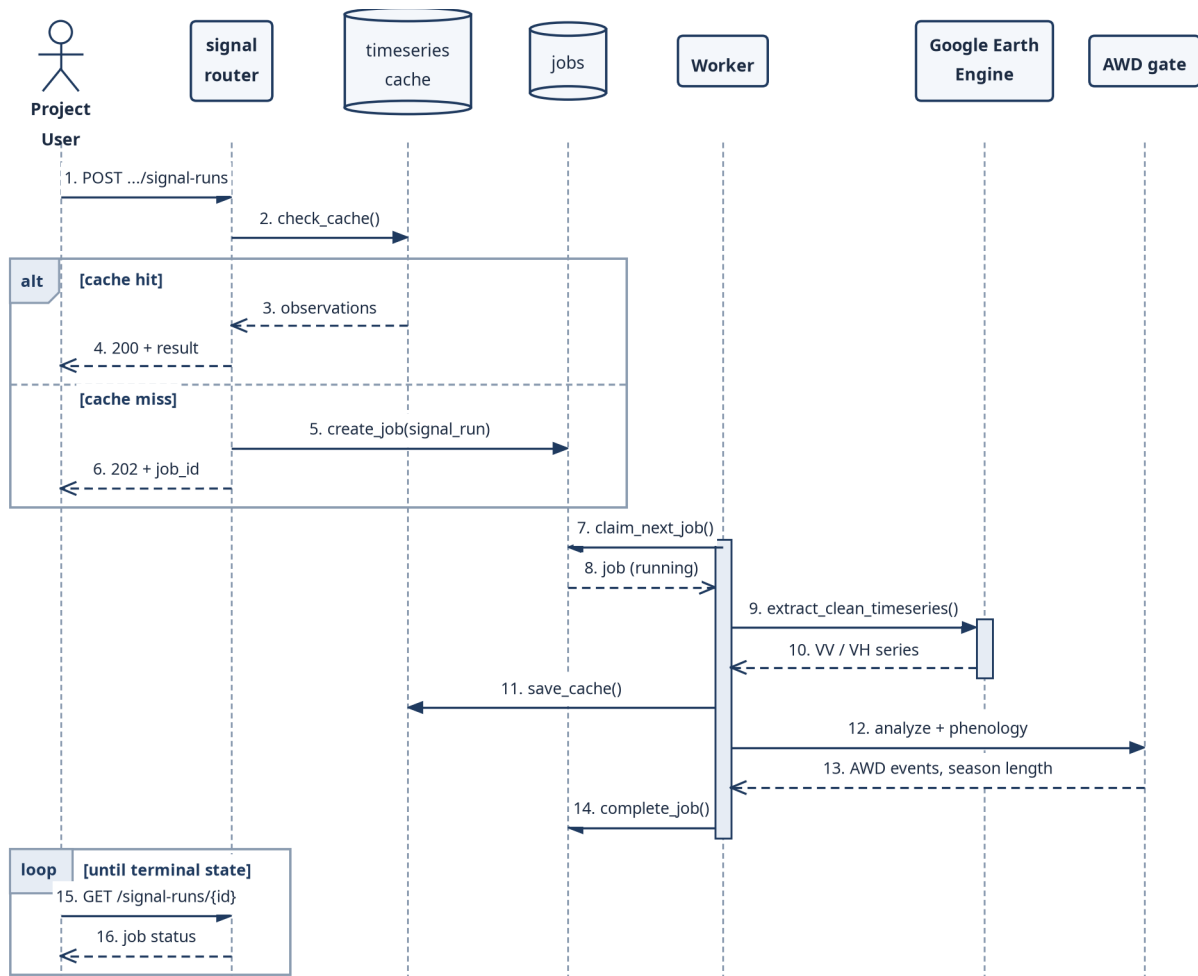


Fig 21: Sequence diagram — Sentinel-1 signal run through the durable job queue.

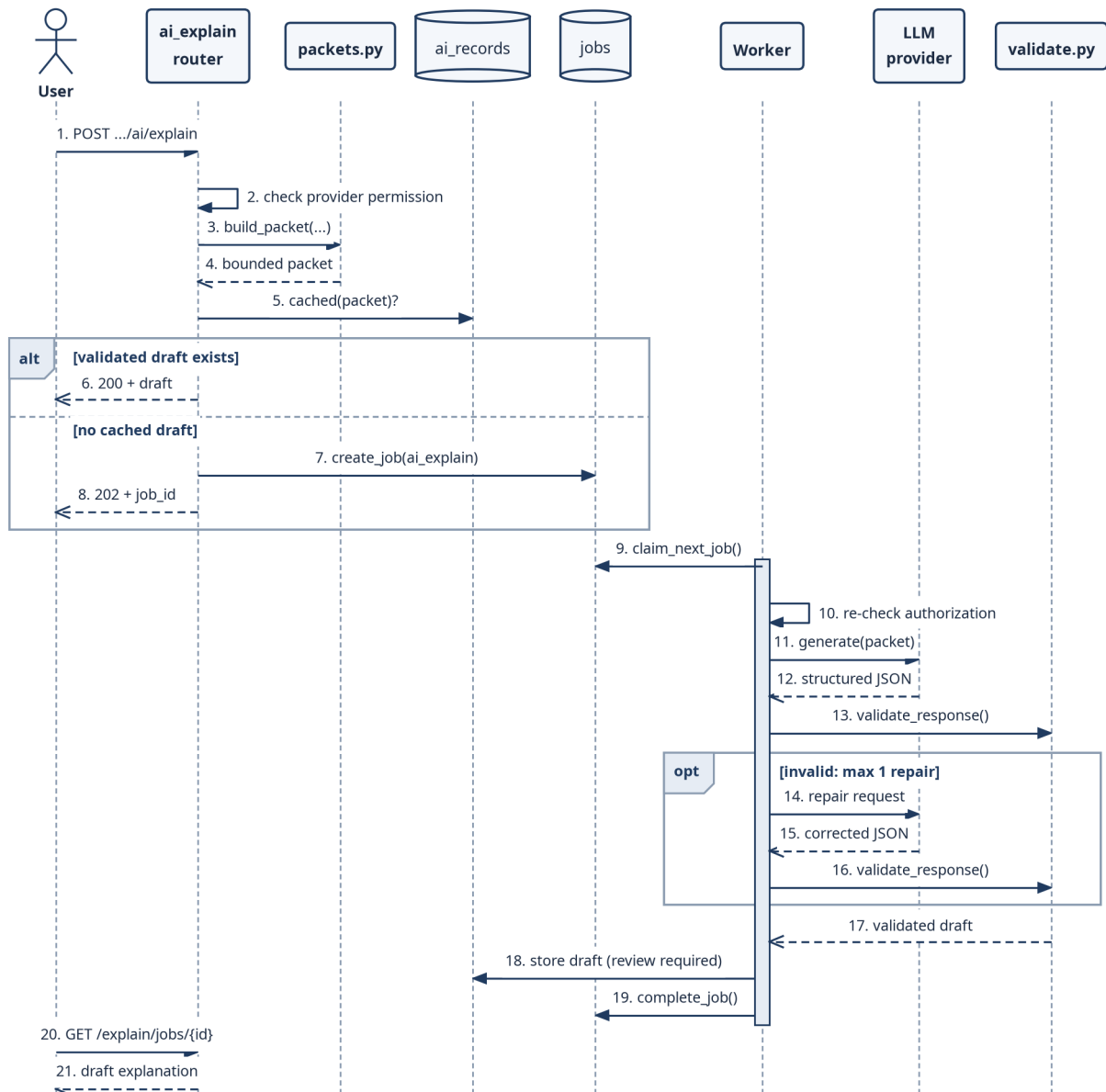


Fig 22: Sequence diagram — bounded AI explanation with server-side validation.

8.4 Deployment Design

Fig 23 shows the deployment architecture. The Next.js frontend is deployed on Vercel and calls the FastAPI backend over HTTPS; the documented production backend runs on Render, and the repository also provides docker-compose (backend, worker and frontend services) and a Railway configuration. Backend and worker run from the same Python 3.12 image: uvicorn serves backend.main:app with a /health endpoint, while python -m backend.worker claims durable jobs from the shared database and calls Earth Engine and AI providers. PostgreSQL is used when DATABASE_URL is set and SQLite otherwise; attachments and model artifacts use local storage or an S3-compatible bucket.

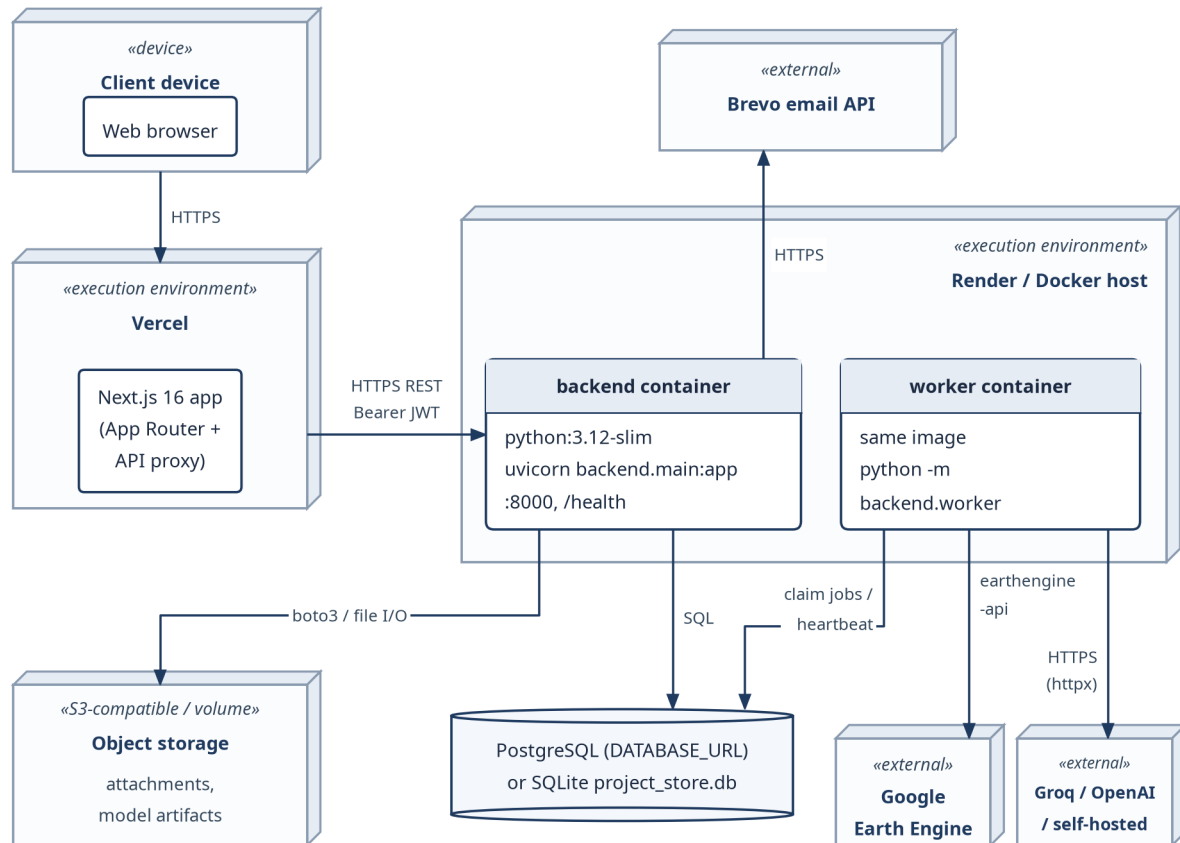


Fig 23: UML deployment diagram of Terra-Audit.

9. AI Engineering Design

9.1 Classical AWD ML Pipeline

The classical AWD ML pipeline sits on the rice signal path; its output is supporting analysis only. It has five parts:

- DatasetBuilder creates training rows from cached rice time series, labelled by the threshold gate. The labels are therefore not independent ground truth.
- Feature engineering turns satellite and time-series attributes into a table, leaving out the gate's own decision flags.
- The shared training harness trains Random Forest [11] (200 trees, class-balanced) and XGBoost [12] (200 estimators, max depth 4) using field-grouped GroupKFold evaluation; at least two independent fields are required and each training fold must contain all label classes.
- A saved model bundle contains the model, its classes and feature names, the satellite processing version and the split strategy. Loading fails if the processing version is out of date, so a model is never applied to satellite features it was not trained on.
- Evaluation reports pooled out-of-fold agreement with the threshold gate: precision, recall and F1, confusion matrices, feature importance and one-vs-rest ROC data. The reports do not call these numbers irrigation accuracy.

9.2 Model Selection and Evaluation

Random Forest and XGBoost are reasonable baselines here because the AWD features are a small tabular set, not raw imagery. Both handle nonlinear interactions, retrain cheaply and report feature

importance. Both are evaluated with the same field-grouped splits, so neither gets an inflated score from seeing the same field in training and validation.

Field grouping fixed one problem from the earlier design, but not the label problem. Because the threshold detector still generates the labels, the metrics only say how well the models copy it. Water-level or irrigation measurements are needed before the ML results can say anything about real AWD events.

9.3 Managed Crop Models

The AI workspace also has crop-model workflows for exploratory crop classification and benchmarking. Unlike the AWD models, they use reviewed crop-identity labels and held-out grouping, but they remain research tools and do not affect the VM0051/VM0042 calculations.

9.4 Generative AI Explanation Architecture

An authenticated user picks one of a fixed set of explanation actions. The server then builds a context packet for that action from the current project and evidence state and from the methodology text in the active bundle. If an equivalent validated explanation is already cached, it is returned; otherwise an `ai_explain` job is queued for the worker. Provider adapters exist for a fake test provider, a self-hosted model, Groq and OpenAI.

Before calling the provider, the worker checks authorization, provider settings and evidence context again. The server validates the structured reply and allows at most one repair attempt. One more authorization and evidence check runs before the explanation is saved, and the user sees it as a draft that a person must review.

9.5 Prompt and Context Design

Prompt design matters here, but the model only sees context that the server chooses. A packet holds the requested action, the active methodology bundle, selected methodology chunks and their corrections, the relevant evidence and calculation data, sentence identifiers, the requirement and record identifiers the model may cite, hashes, and limitations and next actions written by the server. The model cannot invent its own requirement identifiers or change official values. Retrieval is limited to the selected bundle, so an explanation rests on the methodology version and corrections the application picked rather than on the model's general knowledge.

In `tests/test_ai_validate.py` the packet holds one source sentence, "Measured value = 12.3 tCO₂e.", with the identifier `source:s1`, and one readiness row, `vm0042.soc_measurements`, with status missing. The stub provider first answers "999 tCO₂e.". No supplied number matches 999 tCO₂e, so the validator rejects the reply with a `numeric_support` error. `generate_explanation()` then calls the provider a second time and sends the rejected checks along with the packet. The second answer, "12.3 tCO₂e.", cites `source:s1` and passes; it comes back with `retry_count 1` and status `draft_requires_human_review`. If the provider gives the wrong number both times, the function raises `ExplanationValidationError` and no partial result is returned. Other tests in the same file reject a claim that cites an unknown sentence identifier, a record such as `soil_sample:invented` that was not in the packet, and wording such as "compliant", "approved" or "certified".

Beyond these cases, the validator runs fourteen named checks (the `CHECKS` list in `src/ai/validate.py`), covering the schema, sentence, requirement and record membership, required citations, numeric and reference support, prohibited claims, correction context, the scope of missing-evidence entries, server-owned actions and limitations, duplicate requirements and status wording. These checks catch structural errors. They cannot prove that a cited sentence really supports the claim made from it.

9.6 AI Authority Boundary and Evaluation Limitations

Random Forest/XGBoost and the crop models are experimental, and the generative providers only write drafts. None of them can mark a blocked requirement as compliant or change an official number; those decisions stay with the readiness and calculation code (Section 6.3).

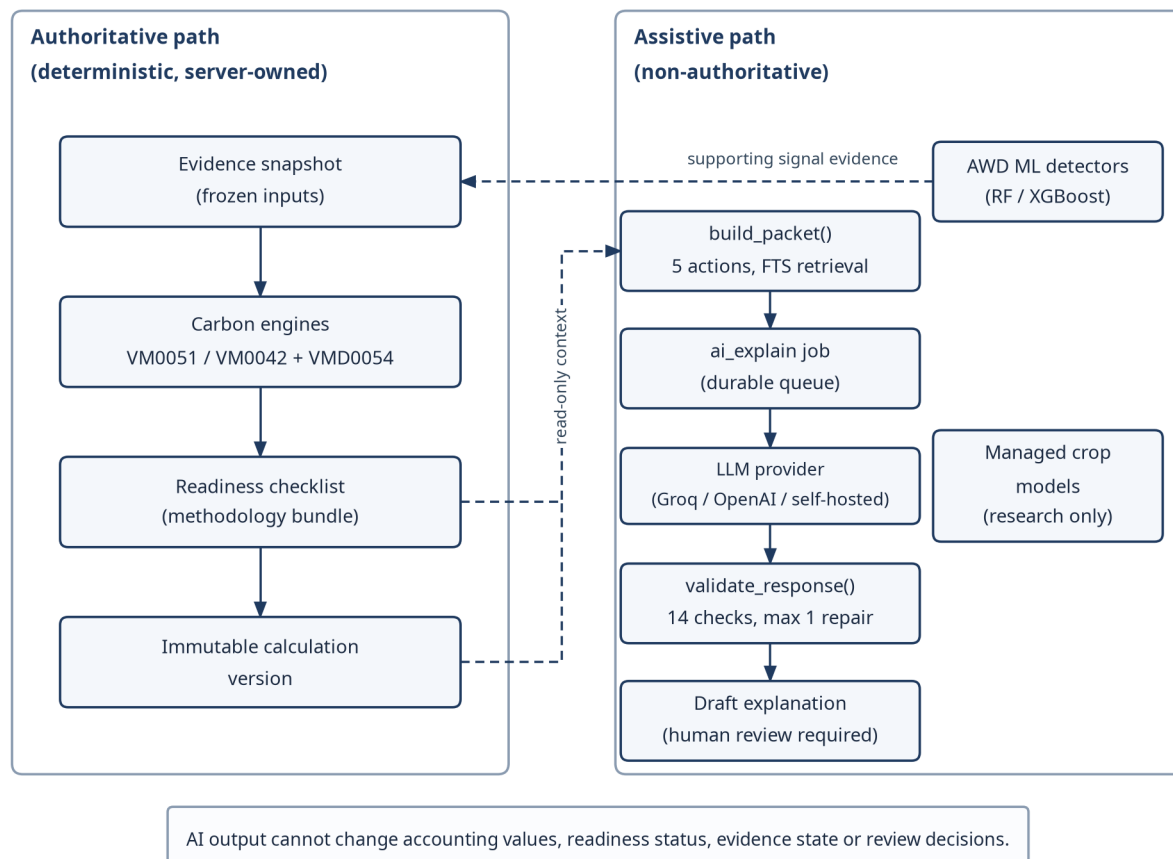


Fig 24: AI authority and data flow: deterministic, server-owned accounting path versus the non-authoritative assistive path.

The regression suite exercises the AI code paths. The planned 20-case evaluation set for generative AI is not finished, because its canonical packet fixtures are still missing, and there is no independent AWD ground-truth dataset yet. Sections 14.3 and 14.6 return to both gaps.

10. Interface Design

Terra-Audit's interface is a Next.js web application organised around projects and fields. A user opens a project, picks a field and moves between its tabs. The tabs depend on the field type: a rice_awd field shows Signal Analytics, while a cropland ALM field shows Practice & Soil Data, Soil Evidence and Production Records instead, so a user never sees a form that does not apply. Starting a Sentinel-1 signal run is a typical action. The request returns 202 with a job ID straight away, and the page polls the job every three seconds (frontend/hooks/use-job-poll.ts) until it is done, failed or cancelled. Each state has its own label, so a failed run does not look like a finished one.

10.1 Interface Objects and Actions

The interface objects come from the user and task analysis in the SRS. Table 14 lists each object, its operations and its route. Available actions depend on the user's role, but the server re-checks authorization on every request.

Table 14: User-interface objects and actions

Interface object	Actions (operations)	Route
Session / account	Sign in, register with OTP, recover access, sign out	/login, /register, /account-access
Project	Create, edit, add members, assign fields, view dashboard	/projects/[projectId]
Field	Register geometry (draw, GeoJSON, KML, coordinates), edit metadata	/fields/new, /fields/[fieldId]/edit
Crop season & evidence	Create season, correct version, add observation / practice event, upload file	/fields/[id]/crop-seasons, /evidence-files
Soil & production evidence	Add sampling plan, sample, lab result, custody event; import production	/fields/[id]/soil-evidence, /production-records
Signal analytics	Choose window and detector, start run, cancel, inspect series	/fields/[id]/signal-analytics
Calculation	Preview, commit (freeze evidence), view version chain, export	/fields/[id]/calculations, /ledger
Methodology readiness	View bundle, applicability and requirement status; record determination	/projects/[id]/methodology
Internal review	Submit, assign reviewer, transition, add finding / comment	/reviews, /reviews/[submissionId]
AI workspace	Request explanation, train / evaluate model, predict	/projects/[id]/ai, /ai-validation
Operations	View queue and worker status, retry / cancel batch, setup	/admin/queue, /admin/setup

10.2 Primary User Workflows

Most work follows one path: select a project and field, build or review evidence, run signal analytics if the field is rice, preview a calculation and save it as a version, check readiness, submit an eligible version to internal review and export whatever artifact is available. Researchers mainly use the signal, ML and AI workspaces; administrators manage methodology sources, provider permissions and worker and job state.

10.3 UI Events and State Transitions

Table 15 maps the most important user events to the visible interface state and the server-owned state behind it (see the state diagrams in Chapter 8).

Table 15: UI events and resulting state changes

User event	UI state before	UI state after	Server-owned state
Start signal analysis	Idle form	Queued – running progress – result chart or error banner	signal_run job pending / running / done / error
Cancel running job	Running	Cancelling – cancelled	cancel_requested – cancelled
Preview calculation	Input form	Preview result with readiness badges	None (no persistence)
Commit (freeze evidence)	Preview shown	Committed version (draft / ready for review) or blocking message	New calculation version
Submit for review	ready_for_review version	Submission listed as submitted	review_submissions = submitted
Reviewer transition /	In review	Status chip and finding list update;	review_events appended

User event	UI state before	UI state after	Server-owned state
finding		approval refused while a blocking finding is open	
Request AI explanation	Idle	Pending → validated draft labelled for human review	ai_explain job; ai_records draft
Acknowledge / resolve issue	Open issue	Acknowledged / resolved with actor and reason	data_quality_issues status
Forbidden or invalid action	Any	Inline error message; entered values kept	403 / 422; no state change

10.4 Responsive, Validation and Accessibility Design

Form controls come from a small set of components in frontend/components/ui. Text inputs, selects and the date picker share one style class, `.ui-control`, which fixes a 44 px height and a visible 2 px focus outline for keyboard users. The date picker sets `aria-invalid` and `aria-describedby` when a value is wrong and closes its calendar on Escape, and the tab groups can be moved through with the arrow, Home and End keys. Status badges carry a text label as well as a colour. Wide content, such as the field tabs and history tables, scrolls inside its own container on narrow screens instead of widening the page. No browser end-to-end test or formal accessibility audit was run on the final version.

10.5 Interface States as Seen by the User

Fig 25–Fig 29 are screenshots of the web interface taken with clearly labelled synthetic demonstration records. They show project navigation, monitoring, calculation readiness and inputs, the review views and the AI workspace. Each caption describes only what can be seen in the screenshot.

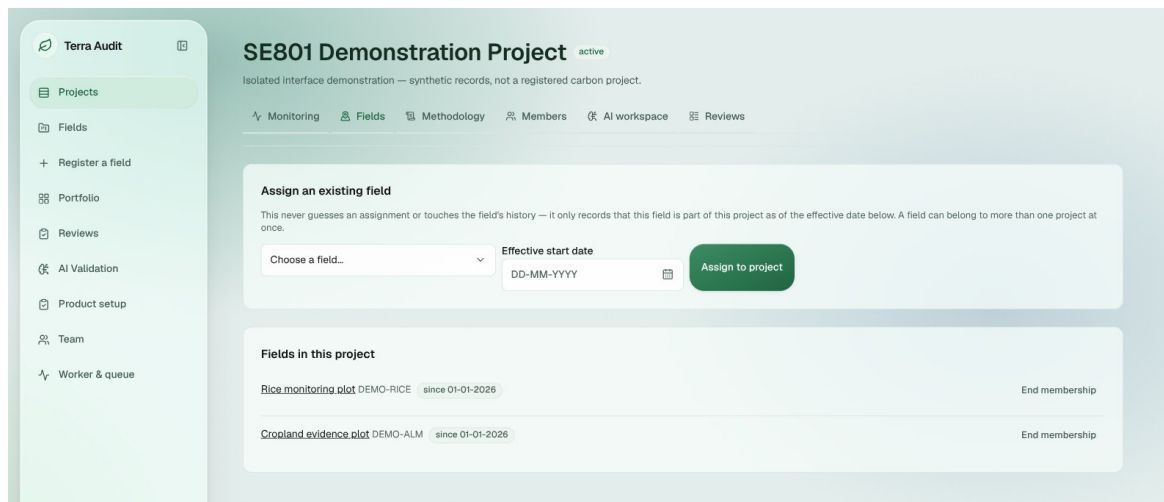


Fig 25: Application shell and project navigation: sidebar, project tabs, field-assignment controls and two demonstration fields.

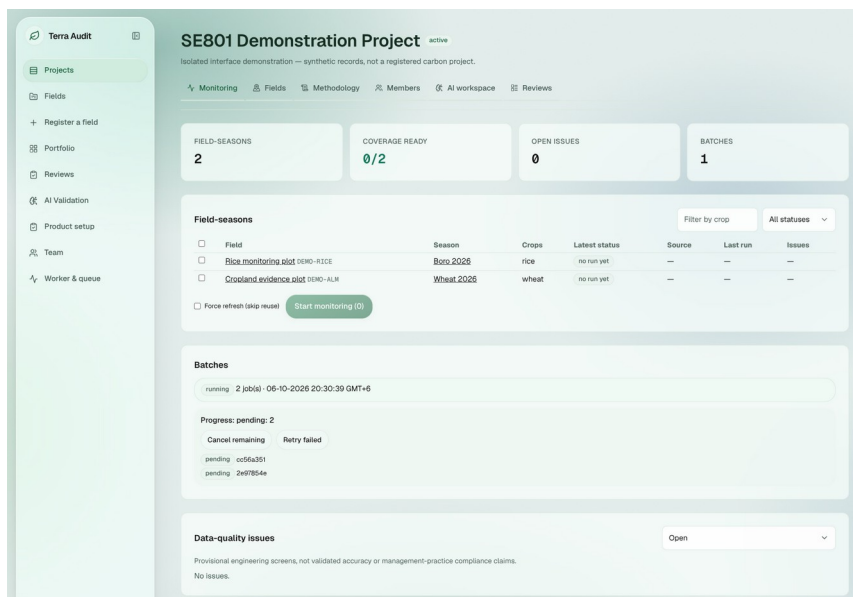


Fig 26: Monitoring batch state: one running batch with two pending child jobs; no data-quality issues.

(a)

Readiness checklist

This reflects what this implementation can check automatically, plus any recorded expert determinations — it is not a certification of full methodology compliance.

needs review	Explain	<code>common.methodology_applicability</code> Declared crop(s) [rice] are within pathway 'vm0051_rice_awd's usual scope per crop taxonomy alone — this is an indicative signal, not a full applicability determination. Actual land-use conditions (e.g. water regime, drainage, prior land use) must still be confirmed by a reviewer before this can be marked satisfied. Required evidence: Field type (immutable at registration) plus the crop(s) declared in linked crop seasons must be within the selected pathway's documented scope.
satisfied		<code>common.monitoring_period_coverage</code> Linked crop seasons cover the requested monitoring period. Required evidence: Crop-season start/end dates spanning the monitoring period.
needs review	Explain	<code>common.evidence_review_status</code> 1 of 1 field observations are not yet accepted by an independent reviewer. Required evidence: Each field observation accepted by a team member other than its author.
missing	Explain	<code>vm0051.required_measurement_inputs</code> AWD event count and season length are the engine's required satellite-derived inputs. VM0051: Improved Management in Rice Production Systems v1.1 §8.2.3 Eq. 6 Required evidence: AWD event count and season length from signal analytics or manual entry. Cannot be manually overridden.
needs review	Explain	<code>vm0051.qa3_pathway_project_size</code> Run a calculation preview to evaluate the QA3 project-size gate. VM0051: Improved Management in Rice Production Systems v1.1 §8.6.3 Required evidence: Computed from the calculation result; not evidence supplied by a user. Cannot be manually overridden.
unsupported	Explain	<code>vm0051.leakage_assessment</code> Not implemented — no evidence can satisfy this in the current codebase. VM0051: Improved Management in Rice Production Systems v1.1 §8.4 Required evidence: Not implemented — no evidence can satisfy this in the current codebase. Implementation: not implemented by this system Cannot be manually overridden.

(b)

Engine inputs

Manual evidence entry for this monitoring period. No measurements are assumed. Use zero only when the evidence records zero; amendment rates may be zero when no material was applied.

Use saved analytics

Only runs matching this field, registered area and both monitoring dates are offered. Confirm that the boundary and selected crop seasons match: older signal jobs do not freeze those versions. Nitrogen, water regime and amendments remain manual.

No compatible run among recent completed analytics. Run analytics for 01-01-2026 to 31-05-2026, or enter documented values manually.

Keep values as manual entry

AWD events (evidence-backed)	Season length (days)
N input (kg N/ha)	Pre-season water regime
	Select documented water regime
Baseline amendment type	Baseline amendment rate (t/ha)
Select documented amendment	
Project amendment type	Project amendment rate (t/ha)
Select documented amendment	

Field area used is always this field's registered area_ha (1.14 ha) — it is frozen from the field record, not re-entered here.

Correct an existing calculation (optional)

New calculation chain

☐ I have checked these values against evidence for the selected period.

Preview calculation Commit (freeze evidence)

Fig 27: Rice calculation readiness and inputs: (a) readiness items marked needs review, satisfied, missing or unsupported; (b) the evidence-backed input form with Preview and Commit (freeze evidence).

(a)

Reviews

Internal review only — this is not external verification or registry issuance.

Notifications
No notifications.

My reviews

Nothing assigned to you
Submissions assigned to you for review will appear here.

Project review queue

SEB01 Demonstration Project

All statuses

- All statuses
- submitted
- in review
- changes requested
- internally approved
- rejected
- withdrawn

No submissions
No submissions have been submitted for review in this project yet.

(b)

Record an evidence review

Project leads and administrators can decide reviewable requirements. Decisions apply to the selected project, dates and current evidence. Changed leakage inputs or production records require a new review.

common.methodology_applicability

Further review needed

Demo evidence requires independent confirmation of water regime and prior land use. No applicability approval has been granted.

Record review

Calculation history

draft v.1 06-10-2026 20:30:39 GMT+6 Estimate: 1.4627 tCO2e

Explain why blocked What changed? Download JSON

Fig 28: Review-related states: (a) the internal Reviews page with its status filter; (b) an evidence-review form and a draft calculation estimate.

Terra Audit

New Project planning

ABC

Monitoring Fields Methodology Members AI workspace Reviews

Multi-crop AI supports monitoring and evidence preparation. Model scores are uncalibrated; outputs require human review and do not authorize carbon credits.

AI provider
groq - openai/gpt-oss-20b. Configuration presence does not verify worker connectivity. Drafts still require human review.

Train a crop model
Requires at least four eligible field-seasons, two crops, and independently reviewed labels. Training uses only active project fields and reserves independent groups for evaluation.

0 eligible seasons - 0 crops - 1 excluded seasons

Version name: September crop baseline Model: Random forest Hold out by: Field

Train and evaluate

Model versions
No trained versions yet.

Active model: Disable predictions Minimum model score: 0.8

Reason for activation or change

Save activation decision

Current revision: 0. threshold: 0.8. Activation records your decision; it does not establish regional validity.

Activation history

Fig 29: AI workspace — crop-model controls: provider label, eligibility count and training/evaluation controls.

11. Implementation

This chapter describes the software as delivered, not its midterm state. Section 11.1 explains how feature status is reported; Section 11.2 works through one calculation per pathway.

11.1 Final Feature Status

Table 19 gives the status of every functional requirement once, next to its verification evidence. “Implemented” means the capability exists and is covered by the architecture; “Partial” means scope, provenance, methodology, authorization or operational limits remain; “Experimental” marks research features that must not be presented as validated accounting evidence. “Automated — pass within final suite” marks an implemented requirement with no recorded limitation and passing tests in the final run.

11.2 Worked Calculation Checks

Rice AWD fixture — Inputs: 2 AWD events, 120-day season, 1.0 ha, 100 kg N/ha, short pre-season water regime, and the default 5 t/ha straw amendment in both scenarios. The engine gives $SC_o = (1+5)^{0.59} = 2.878122$; baseline $CH_4 = 293.568470$ kg and project $CH_4 = 161.462659$ kg, so avoided $CH_4 = 132.105812$ kg = 3.698963 tCO₂e. The QA3 15% deduction is 0.554844 tCO₂e, leaving 3.144118 tCO₂e; the irrigation N₂O term is 0.083210 tCO₂e. Net deterministic estimate = 3.060908 tCO₂e, below the 60,000 tCO₂e/year QA3 gate. This is a code-level worked fixture, not an issued VCU.

Cropland ALM fixture — Inputs follow the fixed 5 ha, one-year regression scenario: baseline synthetic N 120 kg/ha, residue burning 500 kg/ha and diesel 40 L/ha; project synthetic N 80 kg/ha, organic N 20 kg/ha, no residue burning and diesel 25 L/ha; SOC cells are project start [30,32,29], project final [34,35,33], control start [30,31,29], control final [30,31,30]. The mean project-minus-control SOC change is 3.333333 tCO₂e/ha; the independently reconstructed uncertainty is 16.115318%. The engine regression fixture gives $ER = 1.073643$ tCO₂e and uncertainty-adjusted $CR = 13.980780$ tCO₂e; with a 15% non-permanence deduction on the removal term, the deterministic estimate is 12.957306 tCO₂e. The fixture still discloses the separate other-leakage limitation and does not imply external verification or issuance.

Remaining engineering and validation gaps—browser end-to-end tests, PostgreSQL migration, live external services, independent AWD ground truth and external VVB verification—are consolidated with the limitations in Chapter 14.

12. Testing

12.1 Testing Objectives

- Check the deterministic, scoped VM0051 v1.1 and VM0042 v2.2 accounting engines against the factors, equations, golden values, edge cases and hard numerical gates they implement.
- Check that unsupported, incomplete, stale, reviewer-dependent or blocking conditions stay visible and are never turned into “satisfied” or zero.
- Check that new evidence, corrected seasons, recalculation or supersession create new state and leave the stored inputs and results of existing calculation versions untouched.
- Test authentication, organization isolation, project-role checks, private evidence access, account recovery and durable job and worker state transitions across the protected backend workflows.
- Run the frontend tests, lint, TypeScript check and production build, and make sure backend changes do not break the request/response schemas the Next.js application relies on.
- List what the tests cannot show, such as AWD field accuracy, security certification, PostgreSQL migration and live provider reliability, so that a green run is not read as more than it is.

12.2 Testing Approach, Strategy and Pass/Fail Criteria

Tests are layered so that a correct calculation engine cannot hide a bug somewhere else, such as in authorization, evidence storage, background jobs or the frontend contract. Unit and regression tests cover the calculation logic and helpers. API and integration tests run authentication, tenant scoping, evidence, methodology, calculation, export and job workflows against isolated databases. Frontend tests, lint, TypeScript and the production build check the web layer. Deployment and external services are checked manually, since the isolated suite does not reproduce them.

Calculation tests alone would have missed two kinds of bug that occurred during development. The first concerns field-type routing. At one point the ALM endpoints accepted a rice field, and a signal run on a cropland field fell through to the rice SAR detector. The engine tests could not catch this, because they call the engines directly and never pass through the API routing. `tests/backend/test_field_type_guards.py` now sends an ALM practice schedule and SOC values to a rice field, and a signal run to a cropland field, and expects a 422 response each time; the signal test uses a stub that raises if Earth Engine is touched. The same file checks that a user from another organization gets 404 for a field they do not own. The second concerns repeated requests. `test_alm_snapshot_commit_retry_preserves_one_draft_and_external_cumulative_state` sends the same ALM commit twice with one Idempotency-Key and checks that only one calculation row exists, that the second response is marked `already_committed` and that the cumulative ALM state is unchanged. Without this, one user action repeated by the network could create a second version.

A software check passes if it completes with no unexpected failure in the final run: zero failed backend tests, zero failed frontend tests, a successful production build and TypeScript check, and no ESLint errors. Methodology and scientific claims need more than that. A test can show that the code follows a given equation or gate, but not that the methodology is complete, that AWD detection is accurate, that the system is production-secure, or that a VVB or Verra would accept the result.

Test environment: Ubuntu Linux with Python (backend virtual environment, `pytest 8`) and Node.js 22 for the frontend. Backend tests run against isolated temporary SQLite databases with FastAPI's TestClient, fixed fixtures and stubbed external providers, so results do not depend on Earth Engine, AI or email availability.

12.2.1 API Endpoint Verification

The backend suite contains direct FastAPI TestClient coverage for the endpoints in Table 16. Only endpoints with HTTP-level tests are listed; lower-level domain tests are not relabelled as API tests.

Table 16: Direct API endpoint verification

ID	Endpoint	Verified behavior	Test evidence
API-01	POST /auth/login	200 + token for valid credentials; 401 for wrong/unknown credentials.	<code>test_auth_endpoints.py</code>
API-02	GET /auth/me	200 with authenticated organization/role/email; 401 without a token.	<code>test_auth_endpoints.py</code> , <code>test_registration.py</code>
API-03	POST /fields	201 for authorized admin; 403 viewer; 409 duplicate ID; 422 invalid field type.	<code>test_fields_endpoints.py</code> , <code>test_field_type_guards.py</code>
API-04	GET /fields/{field_id}	200 for owning tenant; 404 for another organization.	<code>test_fields_endpoints.py</code> , <code>test_field_type_guards.py</code>
API-05	POST crop-season / GET evidence	201 authorized create; 403 viewer; 422 invalid payload; 404 cross-tenant evidence access.	<code>test_multicrop.py</code>
API-06	POST monitoring-run / GET job	202 durable job, pending before worker dispatch, done after execution, 404 cross-	<code>test_multicrop.py</code>

ID	Endpoint	Verified behavior	Test evidence
		tenant.	
API-07	POST /fields/{field_id}/signal-runs	200 on compatible local cache hit; 202 durable job on cache miss.	test_signal_run_cache_path.py
API-08	POST /fields/{field_id}/calculations	201 with Idempotency-Key and valid ALM context; persists immutable calculation/snapshot state.	test_evidence_verification.py
API-09	GET /calculations/{calculation_id}/evidence/pdf	200 application/pdf for owner, 404 cross-tenant, 403 unauthorized viewer; PDF remains snapshot-backed after live evidence changes.	test_evidence_verification.py
API-10	GET/POST methodology corrections	readable list; 403 viewer confirm; 200 admin confirm; 404 missing resource.	test_methodology_corrections.py
API-11	GET/POST /team/users	403 non-admin; 200 admin list; 201 invite; 409 duplicate email; 422 invalid role.	test_team_router.py

Review, batch, AI-provider and other behavior is also covered by the wider regression and integration suite, but such tests appear in Table 16 only if they call the endpoint through TestClient. Table 18 gives the overall regression status.

12.3 Detailed Test Cases

Table 17 writes out twelve representative automated tests in test-case format with their detailed outcomes. All twelve are part of the backend suite re-run on 7 October 2026.

Table 17: Detailed test cases with outcomes

ID	Test case / precondition	Steps / input	Expected result	Actual result	Status
TC-01	VM0051 QA3 size gate. 2 AWD events, 120-day season, 500,000 ha, 0 kg N	CarbonAssetEngine.calculate_credits()	Reject results above 60,000 tCO2e/yr; no final quantity	qa3_pathway_valid=False; final_issuance=None; reason cites the 60,000 threshold	PASS test_carbon_calculator.py
TC-02	VM0042 without leakage evidence. Valid 5 ha ALM fixture, no frozen leakage_result	AlmCarbonEngine.calculate_credits()	Block instead of assuming zero leakage	production_decline_leakage_blocked=True; final_issuance=None; not issuable	PASS test_carbon_calculator_alm.py
TC-03	Cross-organization isolation. Field in testorg; admin of otherorg	GET /fields; GET /fields/{id}	Other organization cannot list or read the field	Empty list; HTTP 404	PASS test_fields_endpoints.py
TC-04	Snapshot-backed ALM PDF. Committed ALM calculation	Change live field/practice data; GET /calculations/{id}/evidence/pdf as other org, viewer, owner	Unauthorized rejected; PDF reflects the frozen snapshot	404 / 403 / 200 application/pdf; content matches the snapshot	PASS test_evidence_verification.py
TC-05	Durable monitoring job. Authorized crop season	POST monitoring run; worker claims; query from other org	202 pending – running – done; other org denied	Lifecycle asserted; cross-org 404; stored evidence unchanged	PASS test_multicrop.py
TC-06	AI output validation. Packet allowing one sentence and one	Validate invented values/records and prohibited claims;	Invalid output rejected; one repair may	ExplanationValidationError; retry_count=1 succeeds; two failures	PASS test_ai_validate.py

ID	Test case / precondition	Steps / input	Expected result	Actual result	Status
	requirement	one-retry path	succeed	raise	
TC-07	Idempotent commit retry. ALM field with frozen calculation context	POST /fields/{id}/calculations twice with the same Idempotency-Key	One draft version; retry returns it unchanged	201 then 201 with already_committed=True; 1 calculation row, 1 key row; cumulative state unchanged	PASS test_idempotency.py
TC-08	Signal-run cache path. Rice field; cached and uncached windows	POST /fields/{id}/signal-runs for each window	Cache hit answers synchronously; miss enqueues a job	200 with 20 cached observations; 202 with job_id	PASS test_signal_run_cache_path.py
TC-09	Pathway guards. ALM field and rice field	Signal run on the ALM field; ALM practice schedule on the rice field	Pathway-specific endpoints refuse the wrong field type	HTTP 422 'not satellite-driven'; HTTP 422 'requires cropland_alm_vm0042'	PASS test_field_type_guards.py
TC-10	OTP brute-force lockout. Pending registration	Verify five wrong OTP codes, then the correct code	Lock the registration after five failures	400 for each wrong code; correct code then 429	PASS test_registration.py
TC-11	Role and tenant checks on team management. Analyst, viewer, other-org admin	GET /team/users; POST /team/users	Only admins manage the team; roster is org-scoped	403 for analyst and viewer; other admin sees only own org	PASS test_team_router.py
TC-12	Non-issuable result. Rice result with qa3_pathway_valid=False	commit_carbon_credit_result()	Blocked result is never persisted	NonIssuableResultError; credit history count 0	PASS test_issuance_gate.py

12.4 Verification Results

Table 18: Verification results (7 October 2026, commit bc402f2)

Verification area	Command / evidence	Scope	Pass criterion	Result
Backend regression	python -m pytest -q	Full backend/core suite	0 failed	PASS — 177 passed, 0 failed, 0 skipped; 20 warnings; 117 s
Frontend tests	npm test	node --test suite	0 failed	PASS — 4 passed
Production build	npm run build	Next.js production build	Exit 0	PASS
TypeScript	npm run typecheck	tsc --noEmit	Exit 0	PASS
ESLint	npm run lint	Frontend lint rules	0 errors	PASS — 0 errors, 5 warnings
Accounting engines	Backend suite	VM0051, VM0042, VMD0054 gates and golden values	No regression	PASS (in suite)
Methodology / readiness	Backend suite	Bundles, corrections, blocking states	Unsupported stays explicit	PASS (in suite)
Snapshots, auth, jobs, AI validator	Backend suite	Idempotency, tenant guards, queue states, output checks	Covered behavior holds	PASS (in suite)
Generative-AI canonical evaluation	Evaluation runner	20 frozen packet cases	All fixtures run	NOT EXECUTED — fixtures incomplete

Verification area	Command / evidence	Scope	Pass criterion	Result
Browser E2E / accessibility	—	Full user journeys	No critical defect	NOT EXECUTED
PostgreSQL and live services	—	Schema upgrade; Earth Engine / AI / email; worker recovery	Graceful recovery	NOT VERIFIED
Independent AWD ground truth	—	Detections vs field observations	Event-level accuracy	NOT EXECUTED — no field dataset

All five automated checks were re-run on 7 October 2026 against commit bc402f2 of the development branch. The backend suite reported 177 passed, 0 failed and 0 skipped (20 deprecation warnings, 116.98 s); the frontend tests passed 4/4; the Next.js production build and TypeScript check completed; and ESLint reported 0 errors and 5 warnings. The CI workflow should generate Next.js types before its standalone type check; this ordering issue remains open. Browser E2E, PostgreSQL migration, live external services and independent AWD validation were not executed and are reported as limitations.

12.5 Risks and Contingencies

Earth Engine or Sentinel unavailable — Empty results, authentication errors, network failures and API errors end the signal job with an error the user can see. Cached observations cut down repeat calls but do not replace testing against the live service.

Worker unavailable or stale — Jobs are stored in the database, so they are not lost when the API request ends. Heartbeat state and the operations views show when a worker has stopped, and jobs are then retried or cancelled rather than reported as complete.

AI provider unavailable or invalid output — Explanations are optional, so a provider failure does not stop a calculation. A cached validated explanation is reused when one exists. Otherwise the job ends with a visible error, for example when no provider is configured, when the packet is over its token budget (PacketTooLargeError) or when the reply fails validation twice. In none of these cases is a readiness or calculation record written.

Database or storage failure — SQLite suits local and test use and PostgreSQL is supported for hosting, but PostgreSQL migration and restore have not been fully verified. Private evidence and model files need authenticated storage access and a backup plan.

Insufficient satellite observations or weak ground truth — Where permitted, the user can enter explicit manual calculation inputs instead, but missing observations can never be turned into a scientific AWD claim. Independent field validation is still a separate research requirement.

Methodology incompleteness — Unsupported or unresolved requirements stay marked as partial, unsupported, reviewer-dependent or blocking. Gates are not relaxed to make a calculation pass; the calculation stays a draft until the missing requirement is covered by supported evidence or by an external expert or VVB.

12.6 Requirements Traceability Matrix

Table 19 traces each functional requirement to its use case, implementing component, verification evidence and final status. “Pass within final suite” refers to the 177-test backend run; a partial status marks an implementation or assurance limit that remains even when tests are green.

Table 19: Functional requirements traceability matrix

Req.	Use Case / Goal	Component	Verification	Status
FR1	Account access / tenant identity	src/accounts; backend/security.py;	Authentication/account-access tests	Automated — pass within final suite

Req.	Use Case / Goal	Component	Verification	Status
		auth/account-access routers		
FR2	Project & Field Management	src/projects; field/project/team routers; persistence	Project/team/field regression tests	Automated — pass within final suite
FR3	Signal Analytics	src/signals; src/jobs; signal router/worker handler	Signal/cache/job regression tests	Automated — pass within final suite
FR4	Signal Analytics / AWD research	processing.py; threshold_gate.py; src/ai/ml	Signal and ML regression tests	Implemented; software-tested; scientific validation pending
FR5	Evidence & Season Management	src/evidence/ monitoring.py; attachments/monitoring routers	Evidence/monitoring regression tests	Automated — pass within final suite
FR6	ALM Evidence	src/evidence/soil.py; production.py; ALM schedules	Evidence/ALM regression tests	Automated — pass within final suite
FR7	Carbon Calculation — rice	src/carbon/rice.py; calculation/carbon routers	Rice carbon regression tests	Automated — pass within final suite
FR8	Carbon Calculation — ALM	src/carbon/alm.py; leakage_vmd0054.py; snapshot.py	ALM/leakage regression tests	Automated — pass within final suite
FR9	AI Assistance & Research	src/ai/ml; managed-model modules	ML/model regression tests and code review	Implemented/ experimental; independent accuracy pending
FR10	AI Explanation	src/ai/packets.py; providers.py; validate.py; AI job handlers	AI packet/validator regression tests	Implemented/partial; canonical 20-case evaluation pending
FR11	Reporting & Export	src/reporting; backend/routers/export.py	Evidence/snapshot/export regression tests	Implemented/partial; legacy provenance limits remain
FR12	History & Portfolio	calculations/version chain; project/history/portfolio views	Calculation/history/portfolio regression coverage	Implemented; final suite green
FR13	Methodology Readiness	src/methodology/ registry.py; library/corrections/readiness.py	Methodology/readiness regression tests	Implemented/partial methodology coverage
FR14	Calculation Versioning	src/carbon/snapshot.py; calculations.py; idempotency	Snapshot/version/idempotency tests	Automated — pass within final suite
FR15	Internal Review	src/projects/reviews.py; reviews router/UI	Review-route/domain regression tests	Implemented; entry still depends on readiness blockers
FR16	Portfolio / Operations	portfolio/product-ops/ monitoring-ops views and routers	Backend regression suite + operational review	Implemented/partial

Req.	Use Case / Goal	Component	Verification	Status
FR17	Authorization & editable metadata	backend/access.py; deps.py; field/project update paths	Tenant/auth/team/field tests	Implemented/partial; project enforcement not uniform everywhere
FR18	Durable Jobs	src/jobs/queue.py; worker.py; job handlers	Job/multicrop/background-work tests	Automated — pass within final suite

13. User Manual, Deployment and Repository

This chapter is a user manual for the web application, followed by installation, deployment and repository details. It assumes the user's account already belongs to the required organization and project; Earth Engine and an AI provider only need to be configured for the features that use them. The screens mentioned below appear in the Chapter 10 screenshots (Fig 25–Fig 29).

13.1 End-User Workflows

Table 20 gives the step sequence for each end-user task and what the user should check at the end of it. Menu names match the routes in Table 14.

Table 20: End-user workflows

Task	Steps	Check
Sign in and select a project	Sign in, or register with an e-mailed OTP / recover access → open Projects → select or create a project → confirm lead, contributor or viewer membership.	If an action is refused, check the organization, project and role; the backend re-checks every request.
Register a field	From the project, open field registration → enter the field identity and geometry (draw, GeoJSON, KML or coordinates) → choose the pathway rice_awd or cropland_alm_vm0042 → assign the field to the project.	The pathway cannot be changed after registration.
Record evidence	Create crop seasons → add observations, practice events, production records and files; for ALM also soil sampling plans, samples, lab results, custody events, livestock schedules and the leakage assessment.	Use corrections so earlier versions stay visible; never enter zero for missing evidence.
Rice AWD analysis and calculation	Open Signal Analytics → choose the window and detector → start and wait for completed, failed or cancelled → inspect flooding, drydown and phenology → open Calculations, select the saved signal or enter permitted manual inputs → Preview → Commit (freeze evidence).	Signal output is supporting evidence, not validated ground truth; the internal quantity is not an issued VCU.
ALM (VM0042) calculation	On a cropland_alm_vm0042 field record the evidence above → open Calculations; the server freezes a snapshot and runs VMD0054 leakage and VM0042 → commit a new version → export the snapshot-backed PDF.	Missing applicable leakage evidence stays explicit and blocks the result.
Readiness and internal review	Open Methodology → check the bundle, corrections, applicability and each requirement status → submit a ready_for_review version → the reviewer assigns, records findings and comments, and approves, rejects or requests changes.	Drafts cannot enter review; internal review is not VVB validation or Verra verification.
AI explanation	Choose an action (blocked calculation, missing evidence, applicability, leakage or calculation difference) → submit → read the validated draft when the job completes.	The draft needs human review and cannot change readiness, quantities or evidence.

13.2 Export and Troubleshooting

Table 21: Export options and troubleshooting

Situation	What to do
Export	Versioned calculations export as JSON; snapshot-backed VM0042 versions also export a PDF evidence package; legacy verification records keep their PDF/JSON/CSV packages. Prefer snapshot-backed outputs.
Signal job stays pending	Open Worker & queue status; a stale or absent worker prevents queued jobs from completing.
Earth Engine failure	Check Earth Engine authentication and project configuration, and confirm that the window contains Sentinel-1 observations.
Calculation blocked	Inspect readiness and the pathway hard gates; unsupported requirements cannot be bypassed to obtain a ready label.
AI explanation unavailable	Check provider permission and configuration, the worker state and the reported validation or packet error.
Evidence upload fails	Check file size, type and target; private files are downloaded only through authenticated paths.

13.3 Local Installation and Development Setup

A local setup needs Python 3.12-compatible tooling, Node/npm and, for the SAR pipeline, Earth Engine authentication. The minimal steps are:

1. Clone the public repository and check out the required revision/branch.
2. Create and activate a Python virtual environment.
3. Install backend dependencies with `pip install -r requirements.txt`.
4. For signal analytics, run `earthengine authenticate` and `earthengine set_project <GCP_PROJECT_ID>`.
5. Create a project-root `.env` with at least the required JWT secret, frontend public URL and any external-service configuration. `DATABASE_URL` is optional; when omitted, local SQLite is used.
6. Install frontend dependencies with `cd frontend && npm install`.
7. Start the API with `uvicorn backend.main:app --reload`.
8. Start the durable worker separately with `python -m backend.worker`.
9. Start the frontend with `cd frontend && npm run dev`.
10. Run the verification commands in Chapter 12 before treating a modified revision as release-ready.

13.4 Deployment and Public Repository

Public source repository: <https://github.com/knah1d/terra-audit>

The development branch contains the code described in this report (commit bc402f2, verified on 7 October 2026). The root README documents the architecture, setup, run and test commands, and configuration examples avoid embedding secrets.

Production uses Vercel for the Next.js frontend and Render for the FastAPI backend; docker-compose and Railway configurations are also provided. PostgreSQL (Neon-compatible) and S3-compatible storage are supported. Hosting is optional for evaluation: the complete application runs locally with the steps in Section 13.3.

There is no desktop installer: Terra-Audit runs in the browser, and deploying it means running the web frontend, the API, the database and storage, and the separate worker.

14. Evaluation, Limitations and Conclusion

Terra-Audit started as a local, single-user prototype and is now a multi-user web application for preparing agricultural MRV evidence. In my view the most important result is that the accounting and readiness logic is kept apart from the satellite, ML and generative-AI features, even as the system gained multi-user access, structured evidence, background jobs, methodology bundles, calculation versions and internal review. All regression tests pass, but clear gaps remain in methodology coverage, scientific validation, provenance and operations; the sections below set them out.

14.1 Final Evaluation

Three findings stand out. The first is that the calculations can be traced and repeated, but only within the implemented scope. The two worked fixtures in Section 11.2 show where that line falls. The rice fixture ends at 3.060908 tCO₂e, and each step on the way can be checked by hand: the 5 t/ha straw amendment gives SC_o = 2.878122, AWD avoids 132.105812 kg of methane (3.698963 tCO₂e), and the QA3 deduction and the N₂O term are then subtracted. `tests/test_carbon_calculator.py` checks the parts of this chain separately, including the 0.55 true-AWD and 0.71 single-aeration scaling factors and the QA3 gate, which blocks a 500,000 ha project instead of applying the 15% deduction. The ALM fixture ends at 12.957306 tCO₂e, and its 16.115318% uncertainty is recomputed inside the test from the raw SOC samples rather than read back from the engine. What neither fixture can show is that its inputs describe a real field. The 2 AWD events in the rice fixture are typed in, not detected by satellite and confirmed on the ground.

The second finding is that the system blocks results it cannot support, and this is the intended behaviour. The ALM golden-value test runs the same scenario over one and two verification years. With one year the result passes the issuance gate. With two, `soc_uncertainty_annualization_unresolved` is set and `result_is_issuable()` refuses the result, giving a reason that mentions annualization. Returning a number in both cases would make the tool look more complete, but the annualization question is not resolved in the code, so a block is the correct answer. `common.additionality` is handled the same way: it stays unsupported and blocking (Section 14.2).

The third finding concerns the architecture. Moving to Next.js, FastAPI and a separate worker, together with versioned snapshots, made it possible to pin a review to one calculation version and to show which evidence a result was built from. The AI features fit into this design without changing it, since they add drafts and experimental scores outside the calculation path. The cost was time and some unfinished edges. Integration and hardening moved into September and early October (Section 1.6), provenance on the rice and legacy export paths is still incomplete, immutability is enforced by the application rather than the database, and project-membership checks are not applied everywhere. On 7 October 2026 the backend suite passed 177 tests with no failures or skips, and the frontend tests, production build, TypeScript check and lint (0 errors) all passed. What these results leave open is covered in Sections 14.2 to 14.6.

14.2 Methodology and Accounting Limitations

The VM0051 v1.1 implementation covers only the QA3/default-factor rice pathway. Rice leakage and several fertilizer, burning, monitoring and safeguard elements are missing or partial, and there is no independent proof that remotely detected events meet every methodology requirement. For VM0042 v2.2, the implementation covers QA2-style SOC, uncertainty, selected non-SOC and livestock terms and restricted displacement leakage. QA1/process-model quantification, full DSM and sampling, multi-field leakage, mitigation, nonannual reconciliation and complete sequential-period state are outside the verified scope.

The methodology registry keeps bundles, corrections, applicability and requirement states. At present common.additionality is unsupported, blocking and cannot be overridden. Fields such as final_issuance or VCU-style values are internal estimates only. Terra-Audit does not issue credits on the Verra registry, and its internal review is not VVB verification.

14.3 Scientific Validation Limitations

There is no independent AWD ground-truth dataset yet. The Sentinel-1 threshold results and the RF/XGBoost models therefore show agreement with the threshold rule, not field accuracy. The effects of orbit, incidence angle and rainfall, the gaps between passes and the timing of water-level changes have not been validated. The next step is field work: compare detections with water-level tubes, irrigation logs or partner-verified observations, and report event precision and recall, timing error and performance across seasons and locations.

14.4 Provenance and Reproducibility Limitations

Calculation snapshots keep past inputs and results, and the ALM PDF is built from the frozen calculation context. However, the database has no WORM guarantee, packages are not cryptographically signed, and no single provenance record covers every Sentinel scene, geometry revision, detector configuration, attachment hash and Git revision. The existing fingerprints only detect changes within particular subsets of data and are not a signed audit manifest. Some legacy paths still read current or latest context instead of a snapshot.

14.5 Security and Operational Limitations

Authentication, server-derived organization scope, roles, token revocation, private artifact access and job controls are in place. Even so, project-membership checks are not applied consistently, and the database has no general foreign keys or row-level security. Login throttling, malware and content scanning, deletion cleanup, backup and restore, live worker health, the exact deployed revision, the production database and storage, and PostgreSQL migration were not independently verified. No penetration test was carried out and no security certification is claimed.

14.6 Generative-AI Limitations

Generated output is checked for structure, allowlisted references, citation membership, numeric support, corrections and server-owned actions and limitations. That does not prove the explanation is true or that its citations actually support it, so it remains a draft for human review. A runner for the 20-case evaluation exists, but the frozen canonical packet fixtures are incomplete, so no claim is made about provider quality.

14.7 Prioritized Future Work

P0: fix the CI workflow, which runs only for the main branch and type-checks before the Next.js types are generated; close the endpoint authorization gaps; and resolve the additionality/readiness contradiction without weakening the methodology gate, keeping the regression suite green throughout. P1: make rice provenance specific to each calculation and add a tamper-evident evidence manifest; run independent AWD ground-truth validation in Bangladesh or South Asia; verify PostgreSQL, shared storage, worker, backup and deployment recovery; and complete the 20-case AI evaluation. P2: adopt versioned migrations, and extend methodology coverage only where authoritative sources and expert or VVB review support it.

14.8 Conclusion

Terra-Audit shows that the evidence behind an agricultural carbon claim can be collected, run through the supported VM0051 and VM0042 calculations and reviewed in one system, with the calculation kept separate from the AI features around it. Both worked fixtures can be traced from their inputs to the final estimate, and a result that the implemented methodology cannot support is blocked rather than reported. It is not a verification or issuance platform. Field validation of the AWD detector, wider methodology coverage, sealed provenance and production hardening are the next steps, and the external VVB/Verra process remains outside the system.

References

1. Verra, VM0051 v1.1 — Methodology for Improved Management in Rice Production Systems. Repository copy: [methodologies/verra/vm0051/VM0051-Improved-Management-in-Rice-Production-Systems-v1.1.pdf](#).
2. Verra, VM0042 v2.2 — Methodology for Improved Agricultural Land Management. Repository copy: [methodologies/verra/vm0042/VM0042v2.2.pdf](#). The repository also contains the June 2026 correction/clarification file [VM0042v2.2_CC_11JUN2026.pdf](#) and VT0014 v1.0 with its correction notice where relevant to the implemented scope.
3. Verra, VMD0054 v1.1 — Methodology for Estimating Leakage from the Displacement of Agricultural Activities. Repository copy: [methodologies/verra/vmd0054/VMD0054-v1.1.pdf](#).
4. IPCC, *2019 Refinement to the 2006 IPCC Guidelines for National Greenhouse Gas Inventories*, Volume 4 (Agriculture, Forestry and Other Land Use), Chapter 5, Table 5.12 (water regime scaling factors) and Table 5.13/5.14 (pre-season and organic-amendment factors); Chapter 10 (livestock enteric fermentation and manure management, cited for the integrated crop-livestock terms in Sections 1.3, 3.2 and 5.2); Chapter 11, Table 11.1 (N₂O emission factors, including pasture/range/paddock-deposited manure N₂O).
5. IPCC, *Fifth Assessment Report (AR5), Climate Change 2013: The Physical Science Basis* — 100-year Global Warming Potential values (CH₄ = 28, N₂O = 265) used in both carbon estimation engines.
6. Verra, VCS Program Guide; VCS Standard v5.0; and Registration and Issuance Process. Repository copies are stored under [methodologies/verra/standards/](#). These documents are cited only for external program/registration/issuance context; Terra-Audit does not perform registry issuance.
7. European Space Agency / Copernicus Programme, *Sentinel-1 SAR User Guide*. <https://sentinels.copernicus.eu/web/sentinel/user-guides/sentinel-1-sar>
8. Google, *Google Earth Engine API Documentation* (earthengine-api, dataset COPERNICUS/S1_GRD). <https://developers.google.com/earth-engine>
9. R. Lampayan et al., “Adoption and economics of alternate wetting and drying water management for irrigated lowland rice,” *Field Crops Research*, vol. 170, pp. 95–108, 2015 (background on AWD practice).
10. A. Savitzky and M. J. E. Golay, “Smoothing and Differentiation of Data by Simplified Least Squares Procedures,” *Analytical Chemistry*, vol. 36, no. 8, pp. 1627–1639, 1964 (signal smoothing filter used in the data engine).
11. L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
12. T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” *Proceedings of the 22nd ACM SIGKDD*, pp. 785–794, 2016.

13. A. Bouvet and T. Le Toan, "Use of ENVISAT/ASAR wide-swath data for timely rice fields mapping in the Mekong River Delta," *Remote Sensing of Environment*, vol. 115, no. 4, pp. 1090–1101, 2011.
14. D. B. Nguyen, A. Gruber and W. Wagner, "Mapping rice extent and cropping scheme in the Mekong Delta using Sentinel-1A data," *Remote Sensing Letters*, vol. 7, no. 12, pp. 1209–1218, 2016.
15. R. Torres et al., "GMES Sentinel-1 mission," *Remote Sensing of Environment*, vol. 120, pp. 9–24, 2012.
16. N. Gorelick et al., "Google Earth Engine: Planetary-scale geospatial analysis for everyone," *Remote Sensing of Environment*, vol. 202, pp. 18–27, 2017.
17. R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 8th ed. New York: McGraw-Hill Education, 2014.